



# DOSSIER DE PROJET

## CyberAudit & Solutions

*Plateforme web de mise en relation entre PME/TPE et experts en cybersécurité pour la gestion d'audits de vulnérabilité*

**Titre professionnel visé :**

Titre RNCP de Niveau 5 **Développeur Web et Web Mobile**

**Candidat : Tommy JOUHANS**

Rôle sur le projet : Développeur Front-End

Projet réalisé en binôme avec James ROUSSEL (Développeur Back-End)



**Session 2026**

Projet réalisé en formation : du 27 avril au 17 juillet 2026

## Remerciements

Je tiens à remercier l'ensemble des personnes qui ont contribué, de près ou de loin, à la réussite de ce projet portfolio réalisé dans le cadre de ma formation au Titre RNCP de Niveau 5 de Développeur Web et Web Mobile.

Mes remerciements vont en premier lieu à James ROUSSEL, mon binôme sur ce projet, pour la qualité de notre collaboration, la rigueur de nos échanges techniques et la complémentarité de nos compétences Front-End et Back-End tout au long des douze semaines de développement.

Je remercie également l'équipe pédagogique de Holberton School Dijon, notamment mon référent technique et mon relecteur de projet, pour leurs conseils, leur disponibilité et la qualité de leurs retours lors des revues de code et des points d'avancement hebdomadaires.

Enfin, je remercie mes proches pour leur soutien tout au long de cette formation intensive, ainsi que l'ensemble de la promotion pour l'entraide et l'émulation collective qui ont accompagné la préparation de ce dossier.

## Résumé

CyberAudit & Solutions est une plateforme web full-stack destinée à connecter les petites et moyennes entreprises françaises dépourvues d'équipe informatique interne avec des experts en cybersécurité. Selon l'ANSSI, 45 % des PME et TPE françaises n'ont aucune ressource informatique dédiée et restent exposées à des cybermenaces croissantes (rançongiciels, hameçonnage, fuites de données) sans disposer des compétences ni des outils nécessaires pour s'en prémunir.

Développée en binôme sur douze semaines selon une méthodologie agile, l'application permet à un client PME de soumettre une demande d'audit de sécurité en moins de trois minutes, de suivre son avancement à travers quatre statuts, puis de consulter un rapport de vulnérabilités vulgarisé au format PDF, généré automatiquement. Côté administrateur, un expert en cybersécurité gère l'ensemble des demandes et déclenche la génération des rapports.

Le projet repose sur une architecture découplée : React 19/Tailwind CSS 4 côté client, API REST sécurisée en Django 5.1/DRF, PostgreSQL 15 et Celery/Redis pour les tâches asynchrones. La sécurité a été traitée comme une exigence transversale : JWT avec rotation des tokens, RBAC, UUID anti-énumération, protection contre les injections SQL, assainissement XSS et limitation du débit des requêtes.

Ce dossier présente le contexte et le cahier des charges du projet, sa gestion agile, ses spécifications fonctionnelles et techniques, puis les réalisations front-end et back-end argumentées, le jeu d'essai, la stratégie de tests (55 back-end, 56 front-end) et le déploiement continu sur Railway et Vercel.

# Sommaire

|                                                                           |           |
|---------------------------------------------------------------------------|-----------|
| <b>Remerciements .....</b>                                                | <b>2</b>  |
| <b>Résumé.....</b>                                                        | <b>3</b>  |
| <b>Sommaire .....</b>                                                     | <b>4</b>  |
| <b>1. Introduction .....</b>                                              | <b>6</b>  |
| 1.1 Présentation de l'équipe et du contexte de formation .....            | 6         |
| 1.2 Présentation du projet et problématique .....                         | 6         |
| 1.3 Persona et storytelling.....                                          | 6         |
| <b>2. Compétences du référentiel couvertes par le projet .....</b>        | <b>8</b>  |
| <b>3. Contexte du projet et expression des besoins.....</b>               | <b>9</b>  |
| 3.1 Liste des compétences mises en œuvre dans le cadre du projet .....    | 9         |
| 3.2 Expression des besoins .....                                          | 9         |
| 3.3 Contraintes du projet et livrables attendus.....                      | 9         |
| 3.4 Environnement humain et technique.....                                | 10        |
| 3.5 Objectifs de qualité.....                                             | 11        |
| <b>4. Gestion de projet.....</b>                                          | <b>12</b> |
| 4.1 Méthodologie retenue .....                                            | 12        |
| 4.2 Planning et sprints .....                                             | 12        |
| 4.3 Outils de gestion et de communication.....                            | 12        |
| 4.4 Suivi d'avancement, revues et rétrospectives.....                     | 13        |
| 4.5 Gestion des risques.....                                              | 13        |
| <b>5. Spécifications fonctionnelles.....</b>                              | <b>14</b> |
| 5.1 Personas .....                                                        | 14        |
| 5.2 User stories priorisées (méthode MoSCoW) .....                        | 14        |
| 5.3 Maquettes et enchaînement des interfaces.....                         | 15        |
| <b>6. Spécifications techniques.....</b>                                  | <b>26</b> |
| 6.1 Architecture générale en couches.....                                 | 26        |
| 6.2 Stack technique et justification des choix .....                      | 27        |
| 6.3 Flux de données : exemple de la soumission d'une demande d'audit..... | 28        |
| 6.4 Accessibilité (RGAA / WCAG 2.1 niveau AA).....                        | 28        |
| 6.5 Budget de performance (Web Vitals) .....                              | 28        |
| <b>7. Réalisations côté front-end .....</b>                               | <b>30</b> |
| 7.1 Installer et configurer son environnement de travail.....             | 30        |
| 7.2 Maquetter les interfaces utilisateur .....                            | 30        |
| 7.3 Arborescence des composants React.....                                | 31        |
| 7.4 Réaliser les interfaces utilisateur statiques .....                   | 32        |
| 7.5 Développer la partie dynamique des interfaces utilisateur .....       | 32        |

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| 7.6 Sécurité de la partie front-end .....                                        | 34        |
| <b>8. Réalisations côté back-end.....</b>                                        | <b>35</b> |
| 8.1 Mettre en place une base de données relationnelle.....                       | 35        |
| 8.2 Développer des composants d'accès aux données SQL et NoSQL .....             | 38        |
| 8.3 Développer des composants métier côté serveur.....                           | 38        |
| 8.4 Sécurité de la partie back-end.....                                          | 40        |
| 8.5 Documenter le déploiement d'une application dynamique web ou web mobile..... | 41        |
| <b>9. Jeu d'essai .....</b>                                                      | <b>42</b> |
| <b>10. Stratégie de tests et qualité .....</b>                                   | <b>43</b> |
| 10.1 Tests back-end (pytest).....                                                | 43        |
| 10.2 Tests front-end (Vitest et React Testing Library).....                      | 43        |
| 10.3 Tests de sécurité manuels .....                                             | 43        |
| 10.4 Tests d'accessibilité et de performance .....                               | 44        |
| <b>11. Veille technologique et sécuritaire .....</b>                             | <b>45</b> |
| <b>12. Conclusion .....</b>                                                      | <b>46</b> |
| 12.1 Perspectives d'évolution.....                                               | 46        |
| <b>13. Bibliographie et sources .....</b>                                        | <b>47</b> |
| <b>14. Annexes .....</b>                                                         | <b>48</b> |
| Annexe 1 : Glossaire technique .....                                             | 48        |
| Annexe 2 : Correspondance compétences REAC / sections du dossier .....           | 49        |
| Annexe 3 : Matrice des risques complète .....                                    | 50        |
| Annexe 4 : Diagrammes de conception complémentaires.....                         | 51        |
| Annexe 5 : Ressources complémentaires du projet.....                             | 59        |

# 1. Introduction

## 1.1 Présentation de l'équipe et du contexte de formation

Ce projet a été conduit dans le cadre de ma formation au Titre RNCP de niveau 5 « Développeur Web et Web Mobile » à Holberton School Dijon, promotion 2026. Il s'agit d'un projet portfolio réalisé en formation, en binôme avec James ROUSSEL, sur une durée de douze semaines allant du 27 avril au 17 juillet 2026. La répartition des rôles a été clairement établie dès le lancement du projet : j'ai assuré l'intégralité du développement front-end (React, maquettage, interfaces, accessibilité, sécurité côté client), tandis que James ROUSSEL a pris en charge le développement back-end (Django, base de données, API REST, tâches asynchrones).

Cette organisation en binôme, avec des périmètres techniques distincts mais une conception commune, reproduit les conditions réelles d'un projet de développement web en petite équipe, telles que décrites dans le référentiel emploi activités compétences (REAC) du titre : contributions croisées, revues de code obligatoires avant fusion, synchronisation des contrats d'interface entre les deux couches applicatives, et communication continue via des outils collaboratifs (Discord, GitHub, Notion, Trello).

## 1.2 Présentation du projet et problématique

Le point de départ du projet est un constat chiffré et documenté : selon le rapport « Panorama de la cybermenace » publié par l'Agence nationale de la sécurité des systèmes d'information (ANSSI) en 2023, 45 % des petites et moyennes entreprises françaises ne disposent d'aucune équipe informatique interne. Ces structures restent pourtant exposées à des cyberattaques de plus en plus fréquentes et sophistiquées : rançongiciels, hameçonnage ciblé, fuites de données personnelles. Les solutions de cybersécurité existantes sur le marché sont majoritairement conçues pour de grands groupes, avec des coûts et une complexité d'usage qui les rendent inaccessibles à un TPE ou une PME de quelques salariés.

CyberAudit & Solutions répond à cette problématique en proposant une plateforme web qui met en relation simple et directe une PME cliente avec un prestataire d'audit de cybersécurité. Le client soumet une demande d'audit en quelques clics parmi quatre formules (Audit, Security, Protection, Premium), suit l'avancement de son dossier en temps réel à travers un tableau de bord dédié, puis reçoit un rapport de vulnérabilités rédigé en langage clair, accompagné d'un plan d'action sur trente jours. Le tout est délivré dans un environnement sécurisé, avec authentification et séparation stricte des rôles entre client et administrateur.

## 1.3 Persona et storytelling

Pour ancrer les choix de conception dans un besoin utilisateur concret, l'équipe a construit une persona principale, Marie, 45 ans, gérante d'un cabinet comptable de douze personnes à Dijon. Marie utilise Excel et Gmail au quotidien mais n'a aucune notion informatique avancée. Récemment ciblée par un courriel d'hameçonnage qui a bien failli compromettre les données de ses clients, elle cherche une solution simple et abordable pour faire auditer la sécurité de son cabinet, sans avoir à déchiffrer un rapport technique de deux cents pages. C'est cette persona qui a guidé l'ensemble des décisions d'ergonomie du projet : un parcours de soumission de demande d'audit réalisable en moins

de trois minutes, un vocabulaire non technique dans l'interface, et un rapport de vulnérabilités vulgarisé plutôt que brut.

Une seconde persona, Karim, 32 ans, expert en cybersécurité, représente l'utilisateur côté administrateur : il gère la file des demandes reçues, réalise l'audit, saisit les résultats et déclenche la génération du rapport à destination du client.

## 2. Compétences du référentiel couvertes par le projet

Conformément au référentiel emploi activités compétences (REAC) du titre professionnel Développeur Web et Web Mobile, le projet CyberAudit & Solutions couvre l'intégralité des huit compétences professionnelles réparties sur les deux activités-types du titre. Le tableau ci-dessous liste ces compétences et renvoie vers les sections du présent dossier où chacune est mise en œuvre et argumentée.

| Activité-type                                                                   | Compétence professionnelle                                                                   | Section |
|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|---------|
| 1. Développer la partie front-end d'une application web ou web mobile sécurisée | Installer et configurer son environnement de travail en fonction du projet web ou web mobile | 7.1     |
|                                                                                 | Maquetter des interfaces utilisateur web ou web mobile                                       | 7.2     |
|                                                                                 | Réaliser des interfaces utilisateur statiques web ou web mobile                              | 7.4     |
|                                                                                 | Développer la partie dynamique des interfaces utilisateur web ou web mobile                  | 7.5     |
|                                                                                 |                                                                                              |         |
| 2. Développer la partie back-end d'une application web ou web mobile sécurisée  | Mettre en place une base de données relationnelle                                            | 8.1     |
|                                                                                 | Développer des composants d'accès aux données SQL et NoSQL                                   | 8.2     |
|                                                                                 | Développer des composants métier côté serveur                                                | 8.3     |
|                                                                                 | Documenter le déploiement d'une application dynamique web ou web mobile                      | 8.5     |

Les trois compétences transversales de l'emploi type communiquées en français et en anglais, mettre en œuvre une démarche de résolution de problème, et apprendre en continu. Ils sont illustrées tout au long du dossier : documentation technique rédigée en français et en anglais, gestion des blocages techniques rencontrés en sprint (voir section 4.4), et veille technologique et sécuritaire continue (voir section 11).

### 3. Contexte du projet et expression des besoins

Ce projet ayant été réalisé en formation et non au sein d'une entreprise d'accueil, cette section présente, conformément aux exigences du référentiel, la liste des compétences mises en œuvre, l'expression des besoins ayant permis de définir les objectifs et les limites du projet, l'environnement technique retenu, ainsi que les réalisations attendues.

#### 3.1 Liste des compétences mises en œuvre dans le cadre du projet

Le projet a permis de mettre en œuvre l'ensemble des compétences professionnelles des deux activités-types du titre Développeur Web et Web Mobile, détaillées dans le tableau de la section 2, ainsi que les trois compétences transversales de communication, de résolution de problème et d'apprentissage continu.

#### 3.2 Expression des besoins

L'expression des besoins a été formalisée collectivement par l'équipe à partir du constat ANSSI présenté en introduction et affinée à travers la construction du persona Marie. Les besoins fonctionnels retenus sont les suivants :

- Permettre à un client PME de créer un compte et de s'authentifier de façon sécurisée.
- Lui permettre de soumettre une demande d'audit de sécurité en choisissant l'une des quatre formules proposées, en moins de trois minutes.
- Lui permettre de suivre l'état d'avancement de sa demande (en attente, en cours, terminée, archivée).
- Le notifier automatiquement par courriel à chaque changement de statut.
- Lui donner accès, une fois l'audit terminé, à un rapport de vulnérabilités au format PDF, rédigé en langage clair et accompagné d'un plan d'action.
- Permettre à un administrateur expert en cybersécurité de consulter l'ensemble des demandes, de modifier leur statut et de déclencher la génération du rapport.
- Mettre à disposition des clients des modules de sensibilisation à la cybersécurité (hameçonnage, authentification, sauvegarde des données).
- Garantir la sécurité et la confidentialité des données traitées, à chaque étape du parcours utilisateur.

Ces besoins ont ensuite été priorisés collectivement selon la méthode MoSCoW (Must have, Should have, Could have, Won't have), détaillée en section 5.2, afin de définir un périmètre de produit minimum viable réalisable sur douze semaines.

#### 3.3 Contraintes du projet et livrables attendus

Le projet est soumis à une contrainte de délai ferme : douze semaines, calendrier imposé par la formation, avec des jalons hebdomadaires (voir planning détaillé en section 4.2). Il est également

soumis à des contraintes techniques : l'application doit être intégralement sécurisée dès la conception (approche « security by design »), accessible aux personnes en situation de handicap (conformité RGAA visée), et déployée en production avec HTTPS avant la soutenance finale.

Les livrables attendus en fin de projet sont les suivants :

- Une application web fonctionnelle, déployée et accessible publiquement (frontend sur Vercel, backend sur Railway).
- Le code source versionné sur GitHub, avec historique de commits conventionnels et pull requests documentées.
- Une suite de tests automatisés (unitaires et d'intégration) côté front-end et back-end.
- Une chaîne d'intégration et de déploiement continu (CI/CD) opérationnelle.
- Une documentation technique complète (architecture, base de données, API, sécurité, déploiement).
- Le présent dossier de projet et son support de présentation pour la certification RNCP.

### 3.4 Environnement humain et technique

#### *Environnement humain*

| Rôle                  | Personne            | Responsabilité                                                                              |
|-----------------------|---------------------|---------------------------------------------------------------------------------------------|
| Développeur Front-End | Tommy JOUHANS       | Maquettage, interfaces utilisateur, accessibilité, sécurité côté client, déploiement Vercel |
| Développeur Back-End  | James ROUSSEL       | Modélisation des données, API REST, sécurité côté serveur, déploiement Railway              |
| Référent technique    | Formateur Holberton | Accompagnement pédagogique et validation technique hebdomadaire                             |
| Relecteur de projet   | Formateur Holberton | Relecture et retours sur la documentation et le dossier de projet                           |

#### *Environnement technique*

L'environnement de travail retenu s'appuie sur les outils et technologies suivants, détaillés et justifiés en section 6.2 :

- Poste de développement individuel sous environnement de développement intégré (VS Code), avec extensions ESLint, Python et Git.
- Gestion de versions Git, dépôt distant hébergé sur GitHub (dépôt « portfolio ») ;
- Environnement de développement local : Node.js, Python, PostgreSQL, Redis ;
- Outils de conteneurisation Docker pour l'environnement back-end et sa base de données de test ;
- Hébergement de production : Vercel pour le front-end (CDN, HTTPS automatique), Railway pour le back-end, la base de données PostgreSQL et le broker Redis ;

- Outils collaboratifs : Discord pour la communication quotidienne, Notion pour la documentation, Trello pour le suivi des tâches, GitHub Actions pour l'intégration continue.

### 3.5 Objectifs de qualité

Quatre objectifs SMART (Spécifique, Mesurable, Atteignable, Réaliste, Temporellement défini) ont structuré la démarche qualité du projet dès son lancement.

| Objectif                                 | Détail SMART                                                                                                                                                    | Résultat     |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| 1. Authentification et rôles             | Système d'authentification JWT avec deux rôles distincts protégeant toutes les routes sensibles ; 100 % des routes testées via Postman ; livré fin de semaine 3 | Atteint      |
| 2. Gestion des demandes d'audit          | Cycle CRUD complet avec quatre statuts ; cinq scénarios de test validés ; démontrable fin de semaine 4                                                          | Atteint      |
| 3. Déploiement et sécurité en production | Déploiement HTTPS avec protections anti-XSS et anti-injection SQL validées ; URL publique accessible avant la soutenance                                        | Atteint      |
| 4. Expérience utilisateur                | Un utilisateur non technicien (persona Marie) soumet une demande sans aide, en moins de trois minutes, validé par trois tests utilisateurs                      | Atteint (S7) |

## 4. Gestion de projet

### 4.1 Méthodologie retenue

Le projet a été conduit selon une démarche agile inspirée de Scrum, adaptée à une équipe de deux développeurs : douze semaines de développement organisées en sprints hebdomadaires ou bi-hebdomadaires, chacun clôturé par une démonstration (sprint review) et une rétrospective. Cette approche présente l'avantage de livrer des incréments fonctionnels testables à intervalles réguliers, de détecter tôt les écarts entre le planning et la réalité, et de réajuster les priorités à chaque itération. Elle impose en contrepartie une discipline de synchronisation quotidienne entre les deux développeurs, en particulier pour définir les contrats d'interface entre le front-end et le back-end avant de coder de part et d'autre.

### 4.2 Planning et sprints

| Sprint  | Période           | Objectif principal                                                         | Statut  |
|---------|-------------------|----------------------------------------------------------------------------|---------|
| S1      | 27 avril          | Conception et maquettes (Figma), user stories                              | Terminé |
| S2      | 4 mai             | Mise en place de l'environnement (React/Vite, Django/PostgreSQL)           | Terminé |
| S3      | 11-17 mai         | Authentification et sécurité JWT                                           | Terminé |
| S4      | 18-24 mai         | Fonctionnalités cœur 1 : formulaire de demande, tableau de bord client     | Terminé |
| S5      | 25 mai-1 juin     | Fonctionnalités cœur 2 : génération PDF, tableau de bord admin, formations | Terminé |
| S6      | 1-8 juin          | Tâches asynchrones (Celery), notifications, CI/CD                          | Terminé |
| S7-S8   | 8-21 juin         | Tests approfondis, 56 tests front-end Vitest, corrections de bugs          | Terminé |
| S9-S10  | 22 juin-5 juillet | Déploiement Railway (back-end) et Vercel (front-end)                       | Déployé |
| S11-S12 | 6-17 juillet      | Stabilisation et préparation de la soutenance finale                       | Terminé |

Sur les quarante points de complexité planifiés sur l'ensemble du projet, trente-huit ont été livrés, soit une vélocité de 95 %. Les deux points non livrés dans les délais initiaux (tests Cypress de bout en bout) ont été volontairement reportés en fin de projet, la priorité ayant été donnée à la couverture de tests unitaires et d'intégration, jugée plus critique pour la certification.

### 4.3 Outils de gestion et de communication

| Outil          | Usage                                                              | Fréquence            |
|----------------|--------------------------------------------------------------------|----------------------|
| GitHub         | Versionnement du code, pull requests obligatoires, revues croisées | À chaque commit      |
| GitHub Actions | Intégration continue : lint, tests, build à chaque push            | Automatique          |
| Discord        | Communication quotidienne (canaux frontend, backend, général)      | Quotidienne          |
| Notion         | Documentation technique et suivi des tâches                        | Après chaque session |

| Outil              | Usage                                         | Fréquence      |
|--------------------|-----------------------------------------------|----------------|
| Trello             | Tableau kanban de suivi des tâches par sprint | Quotidienne    |
| Point hebdomadaire | Bilan de sprint et replanification            | Chaque semaine |

#### 4.4 Suivi d'avancement, revues et rétrospectives

Chaque sprint s'est conclu par une démonstration des fonctionnalités livrées et une rétrospective structurée autour de trois questions : ce qui a bien fonctionné, les difficultés rencontrées, et les actions à mettre en œuvre pour le sprint suivant. À titre d'exemple, la rétrospective du sprint 2 (tableau de bord et demandes d'audit) a permis d'identifier une difficulté récurrente de synchronisation des interfaces JSON entre le mock front-end et l'API réelle : l'action corrective retenue a été de définir systématiquement les contrats d'interface en amont du développement, dès le sprint suivant, ce qui a nettement fluidifié la suite du projet.

Le format de point quotidien adopté, inspiré du daily stand-up Scrum, a été formalisé par un message asynchrone sur Discord selon la structure : tâches terminées la veille, tâches en cours, blocages éventuels. Cette discipline a permis de détecter rapidement les dépendances bloquantes, par exemple lorsque le développement du tableau de bord administrateur a dû attendre la finalisation du contrôle d'accès par rôle côté back-end.

#### 4.5 Gestion des risques

| Risque identifié                                                      | Probabilité | Impact | Mesure de mitigation                                                                                   |
|-----------------------------------------------------------------------|-------------|--------|--------------------------------------------------------------------------------------------------------|
| Retard sur l'intégration front/back par divergence des contrats d'API | Moyenne     | Élevé  | Définition des schémas JSON en amont (Stage 3), mock localStorage côté front en attendant l'API réelle |
| Faible de sécurité non détectée avant la mise en production           | Moyenne     | Élevé  | Tests de sécurité manuels dédiés (S7-S10), revue de code croisée systématique, veille OWASP            |
| Non-respect du délai de douze semaines                                | Faible      | Élevé  | Découpage en sprints avec priorisation MoSCoW, points hebdomadaires de replanification                 |
| Indisponibilité d'un des deux développeurs                            | Faible      | Moyen  | Documentation continue sur Notion, séparation claire des périmètres front/back                         |
| Dépassement du budget de performance (Web Vitals)                     | Faible      | Moyen  | Lighthouse CI intégré au pipeline, budget de performance défini dès la conception (section 6.5)        |

## 5. Spécifications fonctionnelles

Les spécifications fonctionnelles décrivent le périmètre fonctionnel du projet et les interactions attendues avec l'utilisateur, en amont des phases de maquettage et de développement. Elles se distinguent des spécifications techniques (section 6), qui traduisent ces besoins en solutions d'architecture, de technologies et de sécurité.

### 5.1 Personas

Deux personas ont guidé l'ensemble des choix fonctionnels du projet : Marie, 45 ans, gérante d'un cabinet comptable sans compétence informatique, représentant l'utilisateur client ; et Karim, 32 ans, expert en cybersécurité, représentant l'utilisateur administrateur qui traite les demandes d'audit en parallèle de son activité de conseil.

### 5.2 User stories priorisées (méthode MoSCoW)

Les besoins fonctionnels ont été formulés sous forme d'un user stories et priorisés selon la méthode MoSCoW pour définir le périmètre du produit minimum viable réalisable sur douze semaines.

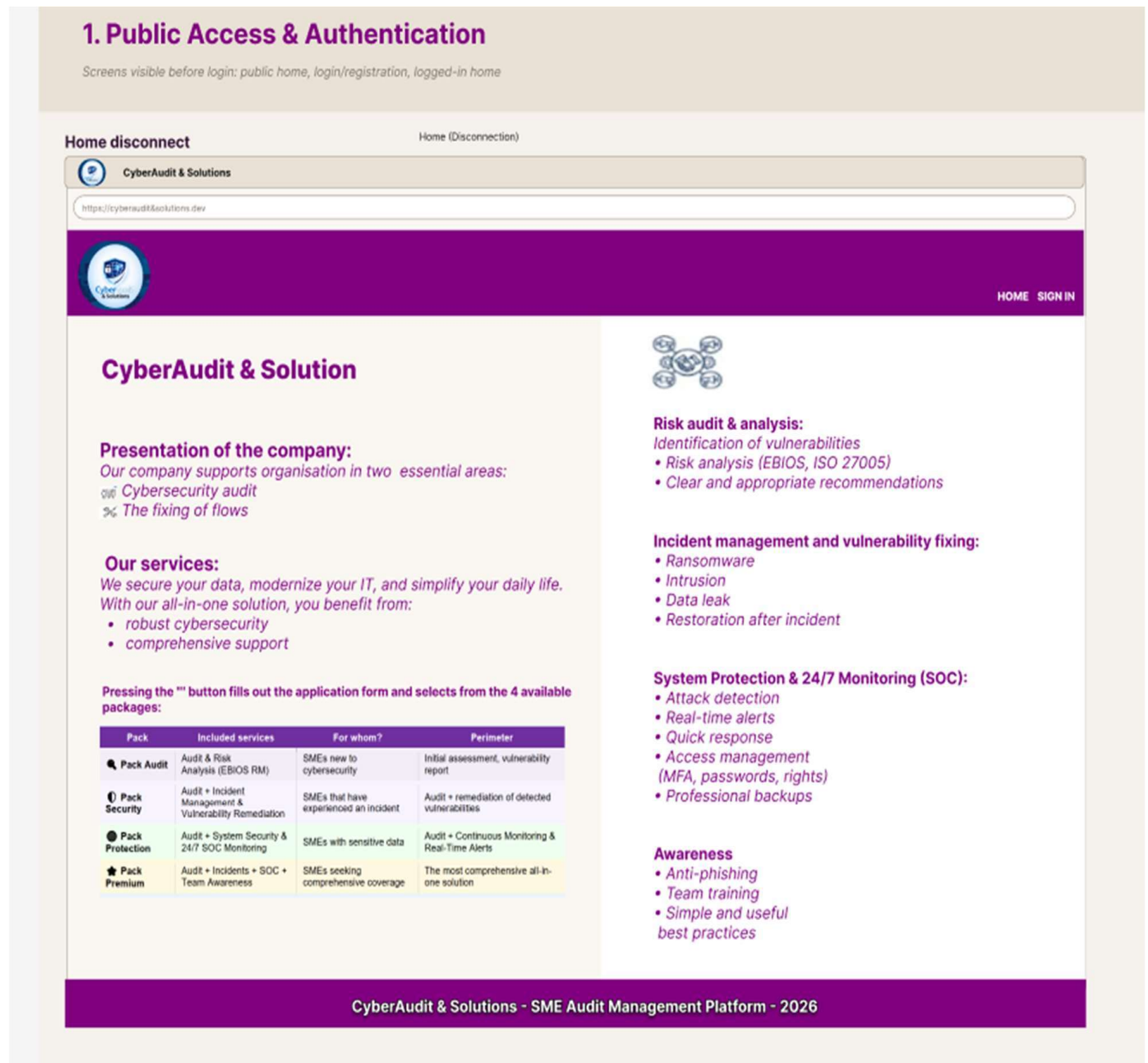
#### ***Must have (fonctionnalités indispensables)***

| Identifiant | User story                                                                     | Sprint |
|-------------|--------------------------------------------------------------------------------|--------|
| US-01       | En tant que visiteur, je veux créer un compte afin d'accéder à la plateforme   | S3     |
| US-02       | En tant qu'utilisateur, je veux me connecter afin d'accéder à mon espace       | S3     |
| US-03       | En tant qu'utilisateur, je veux me déconnecter afin de protéger mon compte     | S3     |
| US-04       | En tant que client PME, je veux soumettre une demande en moins de 3 minutes    | S4     |
| US-05       | En tant que client, je veux suivre le statut de ma demande (4 statuts)         | S4     |
| US-06       | En tant que client, je veux recevoir un courriel à chaque changement de statut | S4     |
| US-07       | En tant qu'administrateur, je veux voir toutes les demandes triées             | S5     |
| US-08       | En tant qu'administrateur, je veux modifier le statut d'une demande            | S5     |
| US-09       | En tant qu'administrateur, je veux générer un rapport PDF                      | S5     |
| US-10       | En tant que client, je veux télécharger mon rapport PDF                        | S5     |

Les user stories de priorité Should have ont porté sur les modules de sensibilisation, la réinitialisation de mot de passe, les filtres administrateur et les notifications visuelles dans l'interface. Les priorités Could have et Won't have (score de sécurité par intelligence artificielle, application mobile native, marketplace de consultants, authentification OAuth2) ont été explicitement exclues du périmètre de la version 1 et documentées comme perspectives d'évolution (voir conclusion, section 12).

## 5.3 Maquettes et enchaînement des interfaces

Onze écrans ont été maquetés sur Figma dès le premier sprint, avant tout développement, conformément à la démarche de conception centrée utilisateur. Chaque maquette a été pensée en priorité pour la persona Marie, avec un objectif constant de simplicité et de minimalité des affichages. Les captures ci-dessous présentent les maquettes principales du parcours client.



Écran 1 : Page d'accueil publique : présentation de l'entreprise, des services et des formules, avec appel à l'action vers l'inscription.

La page d'accueil de la plateforme CyberAudit & Solutions présente l'entreprise, ses services et les packs proposés, taillés pour les entrepreneurs de moins de 50 collaborateurs. Il suffit de se rendre dans l'onglet « SIGN IN » pour que la cliente (Marie) puisse créer un compte et se connecter.

The screenshot shows a web application interface for creating an account. At the top, there's a navigation bar with the company logo and a 'Log in' link. Below this is a purple header with the company logo and 'HOME' and 'SIGN IN' links. The main content area is a light beige color. In the center, there's a white card with a purple header that says 'Create an account' and a sub-header 'Fill in the information below to get started'. The card contains five input fields: 'Company Name' (placeholder: 'Enter company name'), 'First Name' (placeholder: 'Enter first name'), 'Last Name' (placeholder: 'Enter last name'), 'Email' (placeholder: 'Enter email address'), and 'Password' (placeholder: 'Create a password'). Below these fields is a purple button labeled 'Create an account'. Underneath the button is a link that says 'Already have an account?' and a button labeled 'Sign In'. At the bottom of the page, there's a purple footer with the text 'CyberAudit & Solutions - SME Audit Management Platform - 2026'.

Écran 2 : Inscription : e-mail, mot de passe, consentement RGPD/CGU. Règles de validation détaillées en section 7.4.

Après avoir cliqué sur « Créer un compte », la cliente saisit le nom de son entreprise, son prénom, son nom, son adresse e-mail et son mot de passe (haché).

Règles de remplissage du formulaire :

- Chaque champ contient au maximum 50 caractères.
- Si un champ est manquant, le message d'erreur « Veuillez renseigner les informations du formulaire » s'affiche.
- Format de l'adresse e-mail : nom@domaine requis.
- Mot de passe : une minuscule, une majuscule, un chiffre, un caractère spécial, 10 caractères minimum.

The screenshot displays the login interface for the CyberAudit & Solutions SME Audit Management Platform. The page has a light beige background. At the top, there is a purple header bar containing the company logo on the left and the text 'HOME SIGN IN' on the right. Below the header, a central white card with rounded corners contains the login form. The card is titled 'Log In' with a lock icon above it. Below the title, it says 'Enter your credentials to access your account'. There are two input fields: 'Email / User name' with the placeholder 'Enter your email' and 'Password' with the placeholder 'Enter your password'. Below these fields are two links: 'Lost Mail ?' and 'Lost Password ?'. A purple button labeled 'Connection' is positioned below the links. At the bottom of the card, there is a link 'Don't have an account?' and a white button labeled 'Create an account'. The footer of the page is a solid purple bar with the text 'CyberAudit & Solutions - SME Audit Management Platform - 2026'.

Écran 3 : Connexion : e-mail et mot de passe, lien vers la création de compte.

La cliente peut se connecter avec son compte ; sinon, elle clique sur « Créer un compte ».

Les liens « E-mail oublié » et « Mot de passe oublié » sont prévus (à développer ultérieurement pour la présentation RNCP 5).

The screenshot displays the 'Client Portal' of 'CyberAudit & Solutions'. The browser address bar shows 'https://cyberaudit&solutions.dev'. The portal has a purple header with the company logo and 'HOME SIGN OUT' links. A left sidebar contains 'Dashboard', 'Audit Request' (highlighted), and 'Training'. The main area is titled 'Audit request form' and includes fields for 'Username' and 'Company Name'. Under 'Select pack', there are four radio button options: 'Pack Audit' (selected), 'Pack Security', 'Pack Protection', and 'Pack Premium'. To the right, the 'Included Services' section provides details for the selected pack, including a list of services and a price. A 'Message' text area and a 'Sent the audit request' button are at the bottom. The footer reads 'CyberAudit & Solutions - SME Audit Management Platform - 2026'.

Écran 4 : Tableau de bord client : liste des demandes avec statut coloré et bouton flottant de nouvelle demande.

En cliquant sur l'onglet « Demande d'audit », la cliente accède au formulaire de demande d'audit.

Elle doit renseigner :

- Son nom d'utilisateur (50 caractères maximum) Le nom de l'entreprise (moins de 50 caractères)
- Choisir le pack (Audit / Sécurité / Protection / Premium). Lorsque la cliente sélectionne le pack via le bouton radio, les détails du pack s'affichent dans le panneau « Services inclus », notamment « Services inclus », « Pour qui et quels besoins sont adressés ? », « Périmètre d'analyse », « Estimation de l'analyse » et « Tarif ».
- Un message optionnel décrivant la demande d'audit souhaitée.
- Après avoir complété le formulaire, la cliente clique sur « Envoyer la demande d'audit » pour la soumettre.

**Client Dashboard**

CyberAudit & Solutions

https://cyberaudit&solutions.dev

HOME SIGN OUT

**Client Portal**

- Dashboard
- Audit Request
- Training

## My Dashboard

Welcome Marie - Real-time tracking of your audit requests

Open requests  
**2**

Completed  
**1**

Reports available  
**1**

### My audit requests

Filter :

| Case #            | Pack     | Submission | Status      | Action                          |
|-------------------|----------|------------|-------------|---------------------------------|
| DOSSIER-2026-0001 | Premium  | 29/04/2026 | In Progress | <a href="#">View details</a>    |
| DOSSIER-2026-0002 | Audit    | 22/04/2026 | Pending     | <a href="#">View details</a>    |
| DOSSIER-2026-0003 | Security | 10/04/2026 | Completed   | <a href="#">Download report</a> |

### Recent notifications

DOSSIER-2026-0001 - Status changed: Pending → In Progress (2h ago)

**CyberAudit & Solutions - SME Audit Management Platform - 2026**

Écran 5 : Formulaire de demande d'audit en 3 étapes « choix de la formule, périmètre, confirmation. L'objectif est moins de 3 minutes. »

Lorsque la cliente est connectée, elle peut consulter son tableau de bord pour suivre l'avancement de sa demande d'audit (En attente / En cours / Terminé). Elle peut cliquer sur « Voir les détails » et télécharger le rapport via « Télécharger le rapport ».

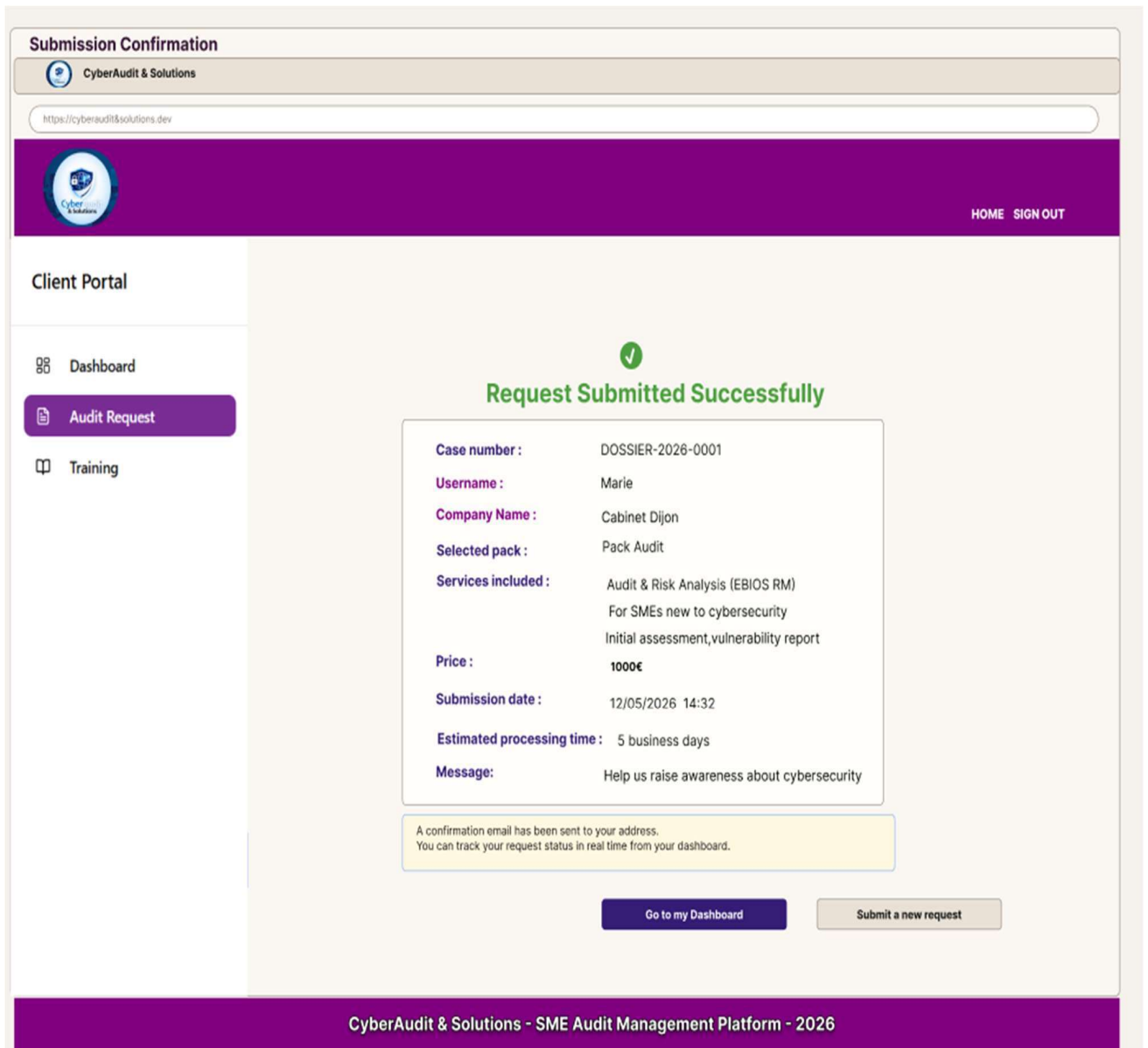
The screenshot displays the 'Client Portal' of 'CyberAudit & Solutions'. The browser address bar shows 'https://cyberaudit&solutions.dev'. The portal has a purple header with the company logo and 'HOME SIGN OUT' links. A left sidebar contains 'Dashboard', 'Audit Request' (highlighted), and 'Training'. The main content area is titled 'Audit request form' and includes fields for 'Username', 'Company Name', and a 'Message'. Under 'Select pack', there are four radio button options: 'Pack Audit' (selected), 'Pack Security', 'Pack Protection', and 'Pack Premium'. To the right, an 'Included Services' box provides details for the selected pack, including a list of services and a price of 1000 €. A 'Sent the audit request' button is at the bottom.

Écran 6 : Détail d'une demande côté client « statut, chronologie, message de confirmation ».

En cliquant sur l'onglet « Demande d'audit », la cliente accède au formulaire de demande d'audit.

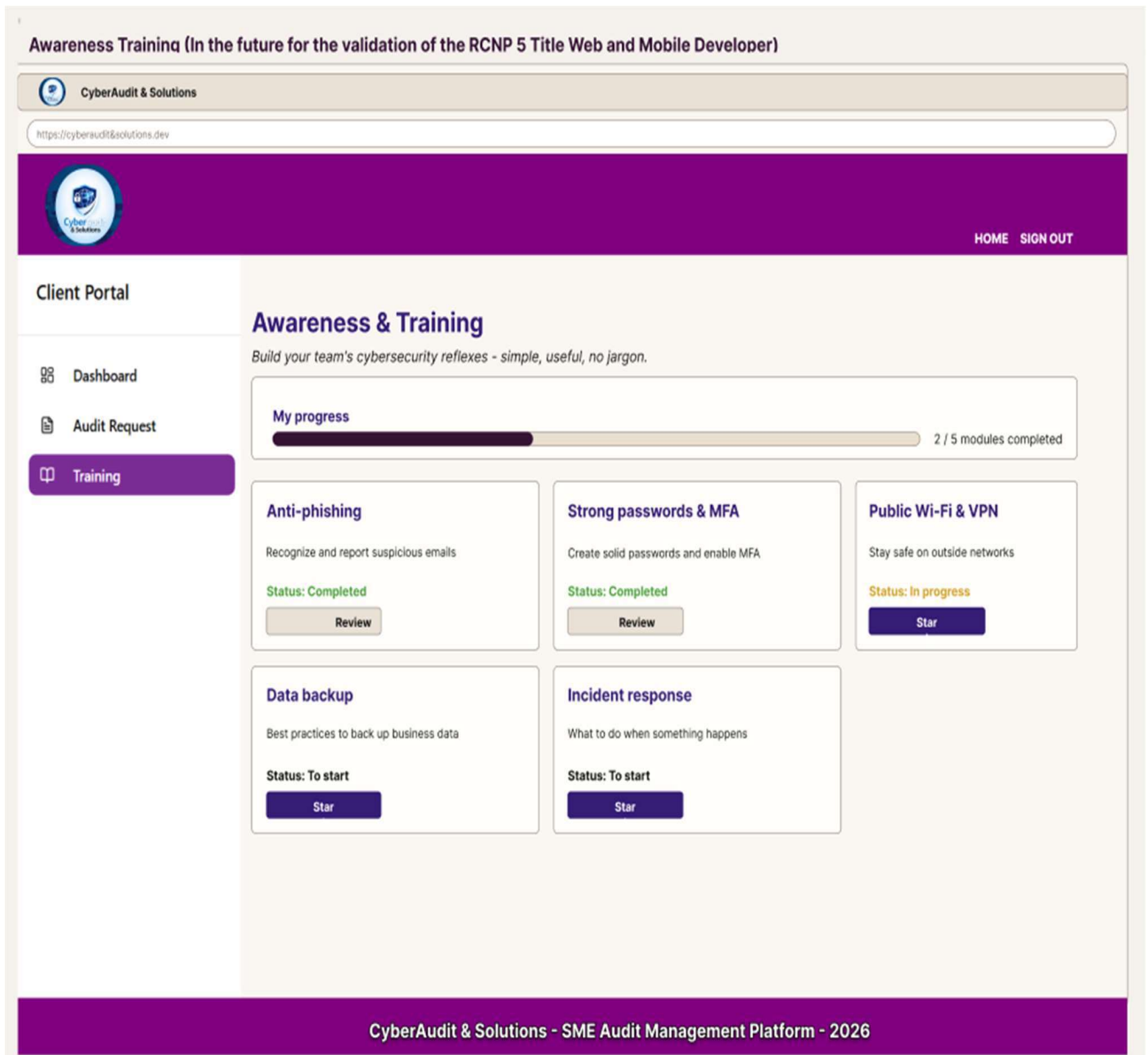
Elle doit renseigner :

- Son nom d'utilisateur (50 caractères maximum) Le nom de l'entreprise (moins de 50 caractères)
- Choisir le pack (Audit / Sécurité / Protection / Premium). Lorsque la cliente sélectionne le pack via le bouton radio, les détails du pack s'affichent dans le panneau « Services inclus », notamment « Services inclus », « Pour qui et quels besoins sont adressés ? », « Périmètre d'analyse », « Estimation de l'analyse » et « Tarif ».
- Un message optionnel décrivant la demande d'audit souhaitée.
- Après avoir complété le formulaire, la cliente clique sur « Envoyer la demande d'audit » pour la soumettre.



Écran 7 : Modules de sensibilisation à la cybersécurité (hameçonnage, authentification multifacteur, sauvegarde des données).

Après l'envoi, un message de confirmation est transmis à l'expert (Administrateur) et un accusé de réception est envoyé à la cliente. La cliente suit l'avancement de l'analyse.



Écran 8 : Tableau de bord administrateur « vue globale des demandes avec filtres par statut et par formule ».

À développer ultérieurement. Modules : « Anti-phishing », « Authentification multi-facteurs », « Connexion publique et Wi-Fi & VPN », « Sauvegarde des données », « Réponse aux incidents ». Sous différents formats (vidéo, visioconférence, documentation) avec une barre de progression. Cette partie de la section sera présentée à l'avenir au-delà du titre RNCP 5 Développeur Web et Web Mobile. L'équipe CyberAudit & Solutions devrait nous aider à préparer les modules de formation dans l'application.

**Admin Dashboard**

CyberAudit & Solutions

https://cyberauditsolutions.dev

HOME SIGN OUT

**Admin Portal**

**Dashboard**

Request

Report

### Admin Dashboard - Audit Requests

CyberAudit operator view - Manage all SME requests

Pending  
**5**

In Progress  
**8**

Completed  
**23**

Archived  
**12**

**Filters :**

Status :  Pack :  Date :  Client :

| Case #            | Client          | Pack       | Submitted  | Status      | Action      |
|-------------------|-----------------|------------|------------|-------------|-------------|
| DOSSIER-2026-0001 | Marie SME       | Premium    | 29/04/2026 | In Progress | View / Edit |
| DOSSIER-2026-0002 | Tech Corp       | Audit      | 22/04/2026 | Pending     | View / Edit |
| DOSSIER-2026-0003 | BlueRoof SARL   | Security   | 10/04/2026 | Completed   | View report |
| DOSSIER-2026-0004 | City Hall Dijon | Protection | 03/04/2026 | In Progress | View / Edit |
| DOSSIER-2026-0005 | Atelier Pro     | Audit      | 25/03/2026 | Archived    | View report |

5 of 48 requests

1 2 3 4 ..

**CyberAudit & Solutions - SME Audit Management Platform - 2026**

Écran 9 : Détail d'une demande côté administrateur « changement de statut, génération du rapport PDF, envoi de notification ».

L'expert (Administrateur) accède à son tableau de bord rassemblant toutes les demandes. Il peut trier la liste selon le pack choisi et les statuts en cours, puis consulter et modifier chaque demande

**Admin Request Detail**

CyberAudit & Solutions

https://cyberaudit&solutions.dev

HOME SIGN OUT

**Admin Portal**

**Request DOSSIER-2026-0001** [Back to list](#)

**Client information**

Company : Marie SME (Cabinet Comptable)  
 Contact : Marie Dupont  
 Email : marie@cabinet-dijon.fr  
 Submitted : 29/04/2026 at 14:32  
 Pack selected : **Pack Audit**  
 Client message : Help us raise awareness about cybersecurity

**Status & Actions**

Current status :   
 Assigned to :   
 Internal notes  
 EBOS RM analysis ongoing. Awaiting client log files.

[Save changes](#) [Generate PDF report](#) [Send notification](#) [Archive request](#)

**History (immutable - GDPR)**

| Date             | Author       | Action                                |
|------------------|--------------|---------------------------------------|
| 29/04/2026 14:32 | Marie Dupont | Request submitted (Pack Premium)      |
| 29/04/2026 14:32 | System       | Confirmation email sent (Celery)      |
| 29/04/2026 16:10 | Karim        | Status changed: Pending → In Progress |
| 29/04/2026 16:11 | Karim        | Internal note added                   |

**CyberAudit & Solutions - SME Audit Management Platform - 2026**

Écran 10 : Aperçu du rapport de vulnérabilités « niveau de gravité, actifs analysés, description, recommandations, plan d'action à 30 jours ».

En cliquant sur « Voir/modifier », l'expert peut consulter la demande de la cliente, changer le statut via le formulaire « Statut & Actions », générer un rapport PDF via « Générer le rapport PDF », envoyer une notification à la cliente « Envoyer une notification » et archiver la demande « Archiver la demande ».

**PDF Report Viewer**

CyberAudit & Solutions

https://cyberaudit&solutions.dev

HOME SIGN OUT

**Admin Portal**

Dashboard

Request

**Report**

**Vulnerability Report - DOSSIER-2026-0001**

Download PDF

**Global security score**

**B+** Acceptable - 3 medium-severity vulnerabilities found 72 / 100

**Executive summary**

EBIOS RM analysis performed on the IS perimeter declared by the client. 3 vulnerabilities identified, none critical. Action plan attached, prioritized over 30 days.

**Vulnerabilities identified**

| Severity | Asset        | Description           | Recommendation                    |
|----------|--------------|-----------------------|-----------------------------------|
| Medium   | Email server | No DMARC policy       | Activate DMARC + SPF + DKIM       |
| Medium   | Wifi router  | Default WPA2 password | Rotate password and enable WPA3   |
| Low      | Workstation  | Outdated antivirus    | Update Defender + central console |

**30-day action plan**

Week 1: DMARC/SPF/DKIM activation  
 Week 2: Wi-Fi rotation + WPA3 migration  
 Week 3: Endpoint update campaign  
 Week 4: Final compliance check

CyberAudit & Solutions - SME Audit Management Platform - 2026

Écran 11 : Téléchargement du rapport depuis le tableau de bord client.

Le rapport peut être téléchargé. Le rapport identifie :

- Le niveau de vulnérabilités (Faible, Moyen, Élevé)
- Les actifs (serveur de messagerie, routeur Wi-Fi, etc.)
- La description et les recommandations
- Un plan d'action est généré avec le nombre estimé de jours.

La cliente peut télécharger le rapport depuis son tableau de bord via le lien

« Télécharger le rapport ».

### Schéma d'enchaînement des maquettes

L'enchaînement des écrans suit un parcours linéaire pour le client : de la page d'accueil publique (écran 1), l'utilisateur non authentifié est dirigé vers l'inscription (écran 2) ou la connexion (écran 3). Une fois authentifié, il accède à son tableau de bord (écran 4), point d'entrée vers la création d'une nouvelle demande (écran 5), la consultation du détail d'une demande existante (écran 6), les modules de formation (écran 7) ou le téléchargement d'un rapport disponible (écran 11 depuis l'écran 10). Côté administrateur, l'enchaînement bascule après authentification vers le tableau de bord global (écran 8), puis vers le détail et le traitement d'une demande individuelle (écran 9), qui déclenche la génération du rapport consulté en écran 10. Ce schéma de navigation a été formalisé et validé collectivement avant le début du développement des interfaces statiques (section 7.4).

## 6. Spécifications techniques

Les spécifications techniques traduisent les besoins fonctionnels en solutions concrètes d'architecture, de technologies et de sécurité. Elles couvrent l'environnement, le matériel, l'hébergement, l'accessibilité et la maintenance de l'application.

### 6.1 Architecture générale en couches

L'application repose sur une architecture en couches découplée, qui sépare clairement la présentation (client React), la logique métier et l'accès aux données (API Django) et la persistance (PostgreSQL, Redis). Ce découplage front/back permet deux déploiements indépendants, un contrôle fin des origines autorisées (CORS), et la mise en cache de l'application front-end statique sur un réseau de diffusion de contenu (CDN).

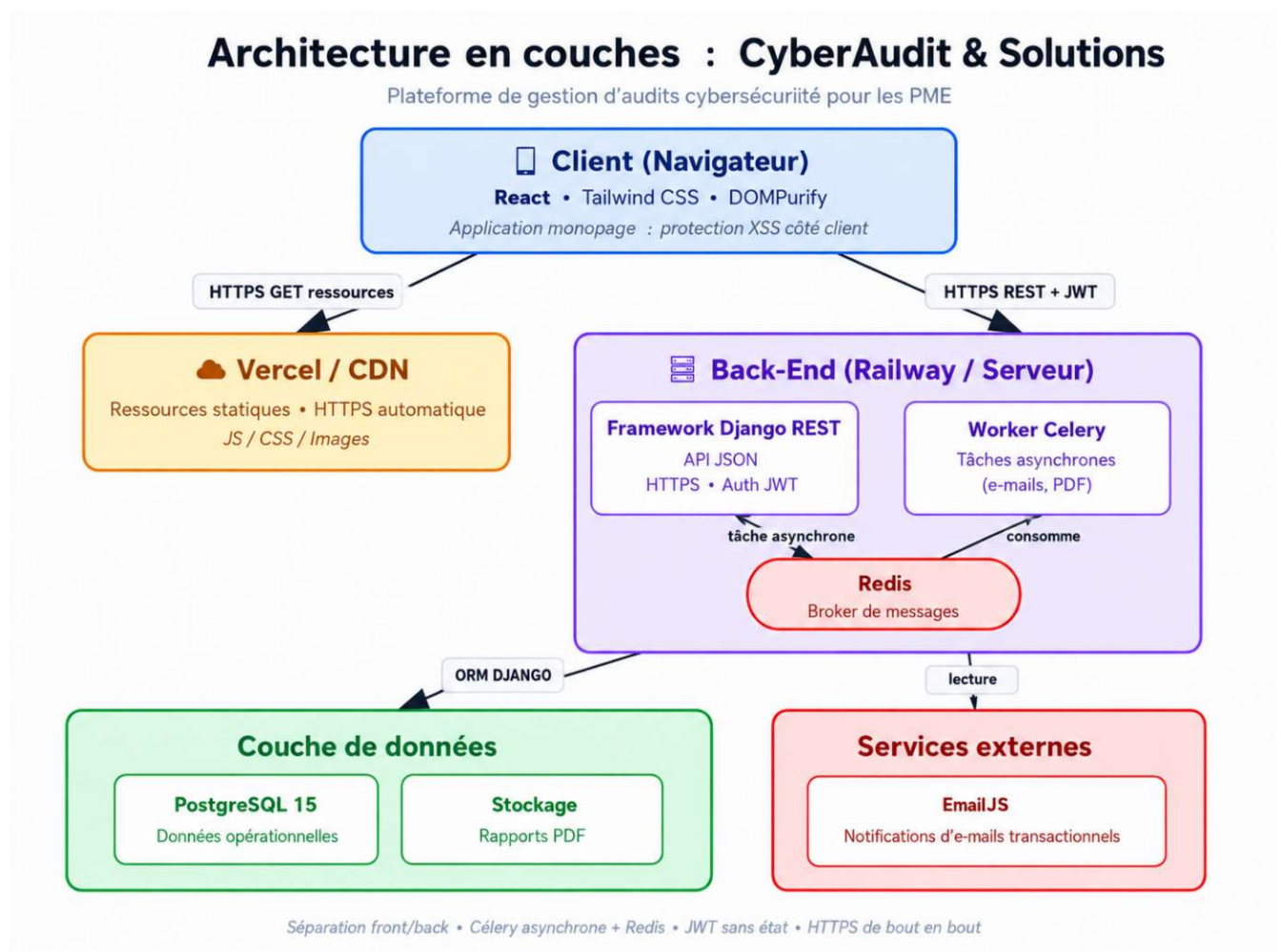


Figure 1 : Architecture en couches de CyberAudit & Solutions : client React/Vite, API Django/DRF, worker asynchrone Celery, bases de données PostgreSQL et Redis.

## 6.2 Stack technique et justification des choix

| Couche                | Technologie                    | Justification                                                                        |
|-----------------------|--------------------------------|--------------------------------------------------------------------------------------|
| Front-end             | React + Vite + Tailwind CSS    | SPA performante, système de conception rapide à mettre en œuvre, écosystème mature   |
| Back-end              | Django + Django REST Framework | Sécurité native (ORM, protection CSRF/XSS), écosystème mature, productivité élevée   |
| Base de données       | PostgreSQL                     | SGBD relationnel ACID, index performants, gestion fine des types (UUID, JSON)        |
| Tâches asynchrones    | Celery + Redis                 | Génération de PDF hors du thread HTTP (2 à 5 secondes), file de messages persistante |
| Authentification      | SimpleJWT                      | Authentification sans état (stateless), scalable horizontalement                     |
| Génération PDF        | WeasyPrint                     | Conversion HTML/CSS vers PDF, réutilisation du système de conception web             |
| Hébergement back-end  | Railway                        | HTTPS automatique, déploiement continu depuis Git, base de données managée           |
| Hébergement front-end | Vercel                         | CDN mondial, HTTPS Let's Encrypt automatique, déploiement continu                    |

Trois décisions techniques structurantes méritent d'être argumentées :

- L'ORM Django plutôt que du SQL brut, qui rend le code intrinsèquement résistant aux injections SQL tout en accélérant le développement grâce au système de migrations ;
- Les clés primaires UUID plutôt que des identifiants entiers auto-incrémentés, qui empêchent l'énumération de ressources par un attaquant (attaque IDOR : /api/audits/1, /api/audits/2...) ;
- Le renvoi d'un code HTTP 404 plutôt que 403 sur les routes réservées à l'administrateur en cas d'accès non autorisé, afin de ne pas révéler l'existence même de ces routes à un utilisateur non habilité.

### 6.3 Flux de données : exemple de la soumission d'une demande d'audit

- Marie renseigne le formulaire dans l'application React (SPA).
- Le front-end envoie une requête POST /api/audits/ au format JSON, avec le jeton JWT dans l'en-tête Authorization: Bearer.
- Django valide la requête via un sérialiseur DRF puis la persiste en base via l'ORM.
- Django place dans la file Redis une tâche Celery asynchrone d'envoi d'un courriel d'accusé de réception.
- Django répond 201 Created avec le détail de la demande créée, incluant sa référence unique.
- Karim (administrateur) voit apparaître la nouvelle demande dans son tableau de bord.

### 6.4 Accessibilité (RGAA / WCAG 2.1 niveau AA)

Le persona principal, Marie, non technophile, impose une exigence forte d'accessibilité. La plateforme vise la conformité au niveau AA du référentiel WCAG 2.1, décliné en France par le Référentiel général d'amélioration de l'accessibilité (RGAA), sur ses quatre principes : perceptible, utilisable, compréhensible, robuste.

| Principe RGAA/WCAG    | Mise en œuvre                                                                                     |
|-----------------------|---------------------------------------------------------------------------------------------------|
| Contraste minimal     | Ratio 4,5:1 sur le texte courant, 3:1 sur le texte large ; vérifié avec axe DevTools              |
| Navigation au clavier | Tous les éléments interactifs accessibles au Tab, ordre de focus logique, anneau de focus visible |
| Étiquettes ARIA       | aria-label sur les badges de statut, role="meter" sur la jauge de score de sécurité               |
| Formulaires           | Chaque champ associé à une balise <label>, erreurs liées via aria-describedby                     |
| Structure sémantique  | Balises header/nav/main/footer, hiérarchie de titres sans saut de niveau                          |
| Images et icônes      | Attribut alt systématique, descriptif ou vide selon le caractère décoratif                        |

L'accessibilité est contrôlée de façon automatisée (axe-core intégré aux tests Vitest, score Lighthouse Accessibilité supérieur ou égal à 90 exigé en intégration continue) et manuelle (navigation clavier complète testée sur les trois parcours critiques, lecteurs d'écran NVDA et VoiceOver).

### 6.5 Budget de performance (Web Vitals)

L'objectif d'un parcours de moins de trois minutes pour un utilisateur non technicien impose une exigence de performance non négociable. La plateforme se fixe un budget de performance aligné sur les indicateurs Core Web Vitals de Google.

| Indicateur                      | Cible              | Signification                                                         |
|---------------------------------|--------------------|-----------------------------------------------------------------------|
| LCP (Largest Contentful Paint)  | < 2,5 s            | Temps d'affichage du plus grand élément visible                       |
| INP (Interaction to Next Paint) | < 200 ms           | Latence de réponse aux interactions utilisateur                       |
| CLS (Cumulative Layout Shift)   | < 0,1              | Stabilité visuelle, absence de sauts de mise en page                  |
| TTFB (Time To First Byte)       | < 600 ms           | Délai serveur, couvre Django et la base de données                    |
| Poids JavaScript                | < 200 ko compressé | Poids initial du bundle JavaScript                                    |
| Score Lighthouse Performance    | ≥ 85/100           | Score agrégé, calculé en intégration continue sur chaque pull request |

Les stratégies d'optimisation retenues incluent le découpage du code par route (React.lazy et Suspense), le tree-shaking automatique via Vite, le chargement différé des images, la mise en cache HTTP des ressources statiques sur le CDN Vercel, ainsi que la pagination systématique des listes côté API (vingt éléments par défaut) et l'usage de `select_related` et `prefetch_related` côté Django pour éviter le problème des requêtes N+1.

## 7. Réalisations côté front-end

Cette section détaille les réalisations personnelles portant sur l'activité-type « Développer la partie front-end d'une application web ou web mobile sécurisée », en argumentant systématiquement les choix de sécurité retenus.

### 7.1 Installer et configurer son environnement de travail

L'environnement de développement front-end a été mis en place dès la première semaine, en tenant compte de l'environnement technique défini dans les spécifications (section 6). L'installation comprend Node.js 20, l'outil de compilation Vite, le gestionnaire de paquets npm, l'éditeur Visual Studio Code avec les extensions ESLint pour le contrôle de qualité de code, ainsi que le client Git pour le versionnement et le travail collaboratif avec James ROUSSEL.

Configuration du projet React/Vite (extrait de package.json) :

```
{
  "name": "frontend",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "test": "vitest"
  },
  "dependencies": {
    "react": "^19.0.0",
    "react-router-dom": "^7.0.0",
    "axios": "^1.7.0",
    "dompurify": "^3.1.0"
  }
}
```

Le contrôle de qualité de code repose sur ESLint (règles React et react-hooks) associé à Prettier pour le formatage automatique, exécutés en pré-commit et vérifiés en intégration continue. Le versionnement suit une convention de commits structurée (Conventional Commits : feat, fix, docs, test, refactor, chore), et chaque fonctionnalité est développée sur une branche dédiée (feature/nom-de-la-fonctionnalité) avant fusion par pull request revue par James ROUSSEL.

### 7.2 Maquetter les interfaces utilisateur

Le maquetage a été réalisé sur Figma à partir du recueil des besoins utilisateurs (section 3.2) et des user stories priorisées (section 5.2). Les onze écrans présentés en section 5.3 respectent une charte graphique cohérente (palette violette #5B21B6, typographie lisible, composants réutilisables), tiennent compte de l'expérience utilisateur du persona Marie et intègrent, dès la conception, les exigences de sécurisation : par exemple, aucun mot de passe n'est jamais affiché en clair dans les maquettes, et les formulaires prévoient systématiquement un espace dédié à l'affichage des messages d'erreur de validation.

Le schéma d'enchaînement des maquettes, présenté en section 5.3, modélise la navigation entre les onze écrans et a été validé avant le début du développement des interfaces statiques, conformément à la démarche attendue par le référentiel.

### 7.3 Arborescence des composants React

L'arborescence ci-dessous illustre la décomposition de l'application en composants réutilisables. Le composant `StatusBadge`, par exemple, est partagé entre le tableau de bord client, le tableau de bord administrateur et la vue de détail d'une demande, ce qui garantit une cohérence visuelle et limite la duplication de code.

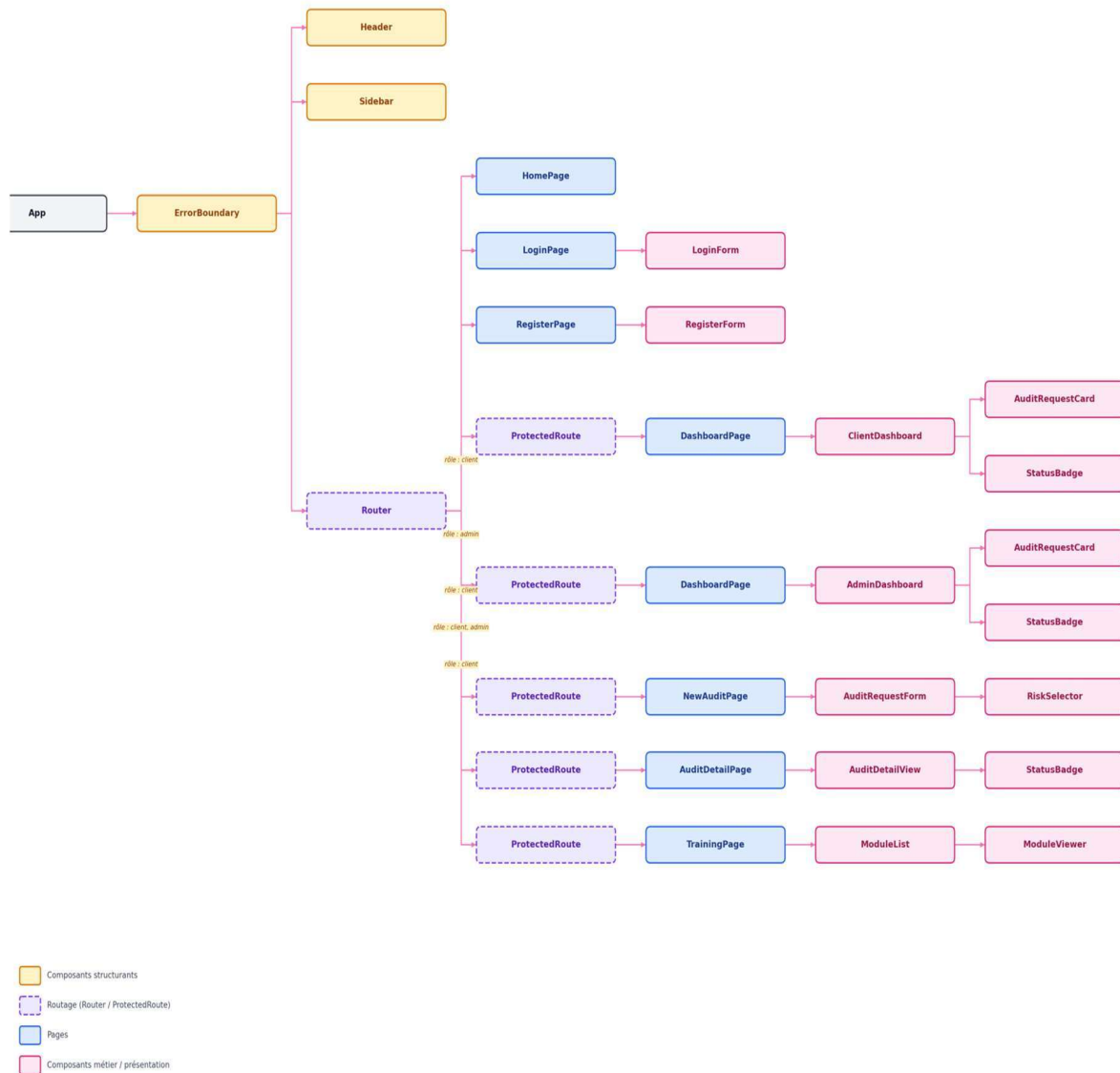


Figure 2 : Arbre des composants React : `ErrorBoundary` englobant le routeur, pages publiques (accueil, connexion, inscription) et pages protégées par rôle (tableau de bord, nouvelle demande, détail, formation).

```
src/
├── components/
│   ├── auth/           LoginForm, RegisterForm, ProtectedRoute
│   ├── dashboard/      ClientDashboard, AdminDashboard, StatusBadge, StatCard
│   ├── audit/          AuditRequestForm, PackSelector, AuditDetailView
│   ├── training/       ModuleList, ModuleViewer
│   └── shared/         Header, Sidebar, Button, ErrorBoundary
├── pages/              HomePage, LoginPage, RegisterPage, DashboardPage, ...
├── hooks/              useAuth, useAudits, useApi
├── services/           api.js, authService.js, dataService.js
└── utils/              sanitize.js, validators.js
```

## 7.4 Réaliser les interfaces utilisateur statiques

Les interfaces statiques ont été développées en HTML sémantique généré par React (JSX) et stylées avec Tailwind CSS 4, en respectant la charte graphique définie sur les maquettes. L'adaptabilité aux différents types de supports (ordinateur, tablette, mobile) a été traitée dès cette étape, à l'aide des classes utilitaires responsives de Tailwind, selon une approche « mobile first ».

Extrait de composant statique : carte de formule d'audit (PackCard.jsx) :

```
export default function PackCard({ pack, selected, onSelect }) {
  return (
    <button
      type="button"
      onClick={() => onSelect(pack.id)}
      aria-pressed={selected}
      className={`w-full rounded-xl border p-4 text-left transition
        sm:w-1/2 lg:w-1/4
        ${selected ? "border-cyber-700 bg-cyber-50" : "border-gray-200"}`
    >
      <h3 className="text-lg font-semibold">{pack.name}</h3>
      <p className="text-sm text-gray-600">{pack.description}</p>
      <span className="mt-2 block text-cyber-700 font-bold">
        {pack.price} EUR
      </span>
    </button>
  );
}
```

Les règles de validation des formulaires statiques ont été précisées dès cette étape, en particulier sur l'écran d'inscription : chaque champ est limité à cinquante caractères, un champ manquant déclenche le message « veuillez compléter le formulaire », l'adresse électronique doit respecter le format nom@domaine, et le mot de passe doit comporter au moins dix caractères avec une minuscule, une majuscule, un chiffre et un caractère spécial.

La documentation du code a été rédigée en français et en anglais selon le contexte, conformément à l'exigence de niveau B1 du CECRL pour la compréhension écrite de l'anglais professionnel. Le référencement des pages publiques (page d'accueil) a été optimisé via des balises meta et un fichier robots.txt, afin d'assurer leur visibilité sur les moteurs de recherche.

## 7.5 Développer la partie dynamique des interfaces utilisateur

La partie dynamique repose sur les hooks React et un contexte d'authentification global (AuthContext), qui gère l'état de connexion, le stockage des jetons JWT et la vérification des rôles. Le composant ProtectedRoute encapsule cette logique pour protéger l'accès aux pages selon le rôle de l'utilisateur connecté.

Extrait : garde d'accès par rôle (ProtectedRoute.jsx) :

```
export default function ProtectedRoute({ children, allowedRoles }) {
  const { user, isLoading } = useAuth();

  if (isLoading) return <Spinner />;
  if (!user) return <Navigate to="/login" replace />;
  if (allowedRoles && !allowedRoles.includes(user.role)) {
    return <Navigate to="/" replace />;
  }
  return children;
}
```

L'appel à l'API REST est centralisé dans une instance Axios unique (api.js), qui injecte automatiquement le jeton JWT dans l'en-tête Authorization de chaque requête authentifiée, et intercepte les réponses 401 pour tenter un rafraîchissement automatique du jeton avant de rediriger vers la page de connexion en cas d'échec. Cet intercepteur illustre une gestion robuste et transparente du cycle de vie de la session côté client, sans intervention de l'utilisateur.

```
// api.js – instance axios centrale avec gestion automatique des jetons JWT
const api = axios.create({ baseURL: BASE_URL });

api.interceptors.request.use((config) => {
  const session = getSession();
  if (session?.access && !isAuthRoute(config.url)) {
    config.headers.Authorization = `Bearer ${session.access}`;
  }
  return config;
});

api.interceptors.response.use(
  (response) => unwrapPagination(response),
  async (error) => {
    if (error.response?.status === 401) {
      return refreshTokenAndRetry(error);
    }
    return Promise.reject(error);
  }
);
```

Le tableau de bord client (ClientDashboard) illustre la partie dynamique la plus représentative de l'application : au chargement, un hook personnalisé useAudits interroge l'API paginée, affiche un indicateur de chargement, puis restitue la liste des demandes avec un composant StatusBadge dont la couleur varie selon le statut (gris pour en attente, bleu pour en cours, vert pour terminer, violet pour archivée). Un clic sur une demande déclenche la navigation vers sa vue de détail sans rechargement de page, grâce à React Router.

## 7.6 Sécurité de la partie front-end

La sécurité côté client a été traitée comme une exigence transversale et non comme une étape a posteriori. Quatre mesures principales ont été mises en œuvre et argumentées :

### ***Protection contre les attaques XSS***

Toute donnée saisie par un utilisateur et réaffichée dans l'interface (par exemple les notes de périmètre d'une demande d'audit, ou le contenu Markdown des modules de formation) est systématiquement assainie par la bibliothèque DOMPurify avant insertion dans le DOM, afin de neutraliser toute tentative d'injection de script malveillant.

```
// utils/sanitize.js – assainissement XSS avec DOMPurify
import DOMPurify from "dompurify";

export function sanitize(value) {
  if (!value) return "";
  return DOMPurify.sanitize(String(value), {
    ALLOWED_TAGS: [], // aucune balise HTML conservée
    ALLOWED_ATTR: [], // aucun attribut conservé
  });
}
```

### ***En-têtes de sécurité HTTP***

Le déploiement Vercel applique des en-têtes de sécurité sur chaque réponse : X-Content-Type-Options: nosniff pour empêcher le navigateur de deviner un type MIME dangereux, X-Frame-Options: DENY pour empêcher tout affichage de l'application dans une iframe tierce (protection contre le clickjacking), une politique de sécurité de contenu (Content-Security-Policy) restrictive, et l'en-tête Strict-Transport-Security (HSTS) qui impose l'usage exclusif du protocole HTTPS.

### ***Gestion sécurisée des jetons d'authentification***

Les jetons d'accès et de rafraîchissement JWT sont stockés en mémoire de session (sessionStorage) plutôt qu'en cookie non sécurisé, et jamais journalisés côté client. Le jeton d'accès a une durée de vie courte de soixante minutes, tandis que le jeton de rafraîchissement, valable sept jours, fait l'objet d'une rotation à chaque utilisation côté serveur (voir section 8.4).

### ***Validation des entrées côté client***

Bien que la validation faisant foi soit systématiquement réalisée côté serveur (section 8.4), une validation côté client est appliquée en amont pour améliorer l'expérience utilisateur et réduire la surface d'exposition de l'API à des requêtes manifestement invalides : le module validators.js centralise les règles de validation (format de l'adresse électronique, robustesse du mot de passe, champs obligatoires) et est couvert par 21 tests unitaires (section 10.3).

## 8. Réalisations côté back-end

Bien que le développement back-end ait été principalement conduit par James ROUSSEL, cette section documente, conformément aux exigences du dossier de projet, l'activité-type « Développer la partie back-end d'une application web ou web mobile sécurisée » telle que conçue et mise en œuvre collectivement pour le projet, les choix de modélisation ayant été définis en binôme lors des sprints de conception (S1-S2), et ma contribution ayant porté sur la définition des contrats d'interface consommés par le front-end ainsi que sur la relecture de sécurité croisée imposée par nos règles de pull request.

### 8.1 Mettre en place une base de données relationnelle

Le schéma conceptuel de données a été élaboré à partir de l'expression des besoins (section 3.2), en identifiant sept entités principales : l'utilisateur (client ou administrateur), la formule d'audit, la demande d'audit, le rapport d'audit, le module de formation, la progression de formation et la notification. Les relations entre ces entités reflètent le cycle de vie métier de l'application : un utilisateur soumet plusieurs demandes, chaque demande référence une formule et donne lieu à au plus un rapport, et chaque notification est rattachée à un utilisateur et le cas échéant à une demande.

S

CyberAudit &amp; Solutions - ER Diagram

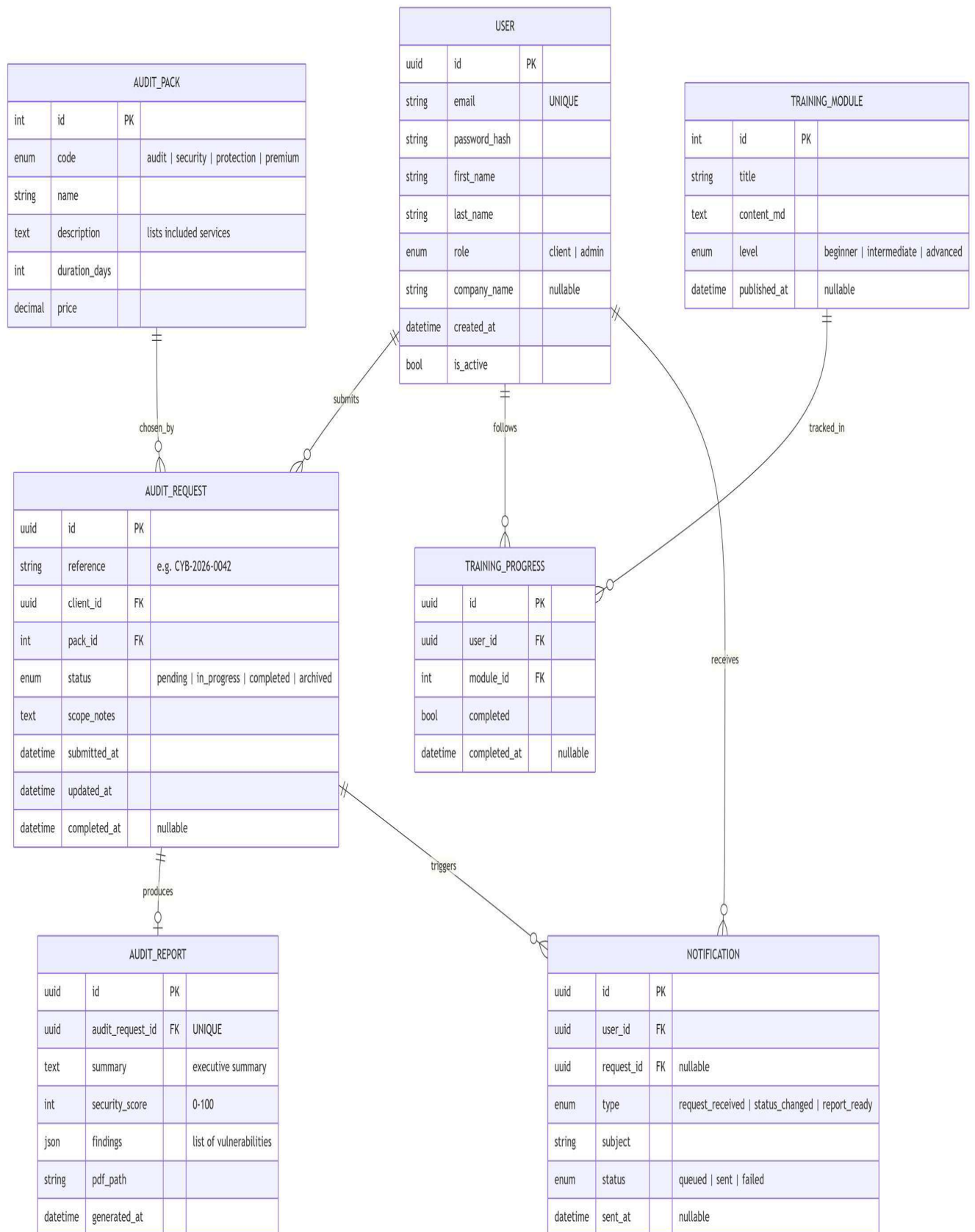


Figure 3 : Schéma physique de la base de données relationnelle PostgreSQL « sept tables, clés primaires UUID, clés étrangères et index de performance ».

| Entité            | Champs clés                                                                  | Relations                                                |
|-------------------|------------------------------------------------------------------------------|----------------------------------------------------------|
| USER              | id (UUID), email (unique), password_hash, role, company_name                 | 1..N vers AUDIT_REQUEST, NOTIFICATION                    |
| AUDIT_PACK        | id, code, name, duration_days, price                                         | 1..N vers AUDIT_REQUEST                                  |
| AUDIT_REQUEST     | id (UUID), reference (unique), status, submitted_at                          | N..1 vers USER et AUDIT_PACK ;<br>1..1 vers AUDIT_REPORT |
| AUDIT_REPORT      | id (UUID), audit_request_id (unique), grade, security_score, findings (JSON) | 1..1 vers AUDIT_REQUEST                                  |
| TRAINING_MODULE   | id, title, content_md, level                                                 | 1..N vers TRAINING_PROGRESS                              |
| TRAINING_PROGRESS | id, user_id, module_id, completed                                            | N..1 vers USER et<br>TRAINING_MODULE                     |
| NOTIFICATION      | id (UUID), user_id, type, status, sent_at                                    | N..1 vers USER et<br>AUDIT_REQUEST                       |

Quatre index ont été ajoutés pour garantir la performance des requêtes les plus fréquentes : un index unique sur l'adresse électronique de l'utilisateur, un index sur le statut d'une demande (filtrage administrateur fréquent), un index composite sur le client et la date de soumission (tableau de bord client), et un index unique sur la relation demande-rapport, garantissant l'unicité d'un rapport par demande.

Script de création de la base de données (extrait de la migration Django initiale des demandes d'audit) :

```
class Migration(migrations.Migration):
    initial = True
    dependencies = [migrations.swappable_dependency(settings.AUTH_USER_MODEL)]

    operations = [
        migrations.CreateModel(
            name='AuditRequest',
            fields=[
                ('id', models.UUIDField(default=uuid.uuid4, editable=False,
                    primary_key=True, serialize=False)),
                ('reference', models.CharField(blank=True, db_index=True,
                    max_length=20, unique=True)),
                ('status', models.CharField(choices=[('pending', 'Pending'),
                    ('in_progress', 'In Progress'), ('completed', 'Completed'),
                    ('archived', 'Archived')], db_index=True,
                    default='pending', max_length=20)),
                ('scope_notes', models.TextField(blank=True)),
                ('submitted_at', models.DateTimeField(auto_now_add=True)),
                ('client', models.ForeignKey(on_delete=models.PROTECT,
                    related_name='audit_requests', to=settings.AUTH_USER_MODEL)),
                ('pack', models.ForeignKey(on_delete=models.PROTECT,
                    related_name='requests', to='audits.auditpack')),
            ],
            options={
                'db_table': 'audit_requests',
                'ordering': ['-submitted_at'],
                'indexes': [models.Index(fields=['client', '-submitted_at'])],
            },
        ),
    ]
```

## 8.2 Développer des composants d'accès aux données SQL et NoSQL

L'accès aux données relationnelles est intégralement réalisé via l'ORM (Object-Relational Mapping) de Django, qui génère des requêtes SQL paramétrées et protège nativement l'application contre les injections SQL, aucune requête brute n'étant utilisée dans le projet. Le modèle User illustre ce composant d'accès aux données, avec un gestionnaire personnalisé (UserManager) qui centralise la création des comptes et le hachage des mots de passe :

```
class UserManager(BaseUserManager):
    """Gestionnaire utilisant l'adresse e-mail comme identifiant principal."""

    def create_user(self, email, password=None, **extra_fields):
        if not email:
            raise ValueError("L'adresse email est obligatoire.")
        email = self.normalize_email(email)
        user = self.model(email=email, **extra_fields)
        user.set_password(password)      # hachage PBKDF2, jamais de clair
        user.save(using=self._db)
        return user

class User(AbstractBaseUser, PermissionsMixin):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    email = models.EmailField(unique=True)
    role = models.CharField(max_length=10, choices=Role.choices,
                           default=Role.CLIENT)
    is_active = models.BooleanField(default=True)
    objects = UserManager()
    USERNAME_FIELD = "email"
```

Au-delà du modèle relationnel, le projet exploite une base de données NoSQL de type clé-valeur : Redis, utilisée comme courtier de messages (broker) pour la file de tâches asynchrones Celery. Chaque tâche différée (génération de rapport PDF, envoi de courriel) est sérialisée et placée dans une file Redis, consommée par un worker Celery indépendant du processus web principal, ce qui illustre la compétence de développement de composants d'accès à une base NoSQL en complément de l'accès relationnel.

## 8.3 Développer des composants métier côté serveur

La logique métier est séparée des modèles de données et regroupée dans des modules de services dédiés, selon le principe de responsabilité unique. Le modèle AuditRequest expose par exemple une méthode métier de génération de référence unique, garantissant l'absence de collision même en cas de créations concurrentes grâce à un verrou de ligne (select\_for\_update) :

```
@classmethod
def _generate_reference(cls):
    """Génère DOSSIER-YYYY-NNNN de façon atomique (anti-collision)."""
    year = datetime.now().year
    prefix = f"DOSSIER-{year}-"
    with transaction.atomic():
        last = (
            cls.objects.select_for_update()
            .filter(reference__startswith=prefix)
            .order_by("-reference")
            .first()
        )
        next_num = (int(last.reference.split("-")[-1]) + 1) if last else 1
    return f"{prefix}{next_num:04d}"
```

Le composant métier de génération du rapport PDF illustre le traitement asynchrone côté serveur : à la demande de l'administrateur, une tâche Celery restitue le gabarit HTML du rapport avec les données de la demande, le convertit en PDF via WeasyPrint, l'enregistre, puis déclenche l'envoi d'une notification par courriel au client. Une politique de nouvelle tentative automatique (jusqu'à trois essais, avec un délai croissant de dix, trente puis quatre-vingt-dix secondes) sécurise ce traitement contre les échecs transitoires.

```
@shared_task(bind=True, autoretry_for=(Exception,), max_retries=3,
             retry_backoff=True)
def generate_pdf_task(self, report_id):
    report = AuditReport.objects.select_related("audit_request").get(id=report_id)
    html = render_to_string("reports/report.html", build_context(report))
    pdf_bytes = HTML(string=html).write_pdf()
    report.pdf_path = save_pdf(pdf_bytes, report.audit_request.reference)
    report.generated_at = timezone.now()
    report.save(update_fields=["pdf_path", "generated_at"])
    send_status_change_email.delay(report.audit_request_id, "report_ready")
```

## 8.4 Sécurité de la partie back-end

### *Authentification et gestion des sessions*

L'authentification repose sur SimpleJWT : un jeton d'accès de courte durée (soixante minutes) et un jeton de rafraîchissement de sept jours, avec rotation systématique du jeton de rafraîchissement à chaque utilisation et mise en liste noire (blacklist) du jeton précédent, afin d'empêcher sa réutilisation en cas de vol.

```
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=60),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
}
```

### *Contrôle d'accès par rôle (RBAC)*

Deux niveaux de défense protègent les routes sensibles : côté front-end, le composant ProtectedRoute (section 7.5) empêche l'affichage d'une page non autorisée ; côté back-end, des classes de permission DRF personnalisées (IsAdmin, IsAdminOrOwner) sont vérifiées sur chaque vue avant tout traitement. Un client tentant d'accéder à la demande d'un autre client reçoit une réponse 404 plutôt que 403, afin de ne pas confirmer l'existence de la ressource à un utilisateur non habilité.

```
class IsAdminOrOwner(BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.user.role == "admin":
            return True
        return obj.client_id == request.user.id
```

### *Limitation du débit (rate limiting)*

Les points d'entrée sensibles aux attaques par force brute (connexion, réinitialisation de mot de passe) sont limités à cinq requêtes par minute et par adresse IP, à l'aide du décorateur django-ratelimit, ce qui neutralise les tentatives automatisées de découverte de mot de passe.

### *Protection contre les injections et validation des entrées*

L'usage exclusif de l'ORM Django élimine par construction le risque d'injection SQL. Chaque requête entrante est validée par un sérialiseur DRF avant tout traitement métier, avec des contraintes explicites sur les types, longueurs et formats attendus. Les champs de texte libre (notes de périmètre, notes internes) ne sont jamais interprétés comme du HTML côté serveur, ce qui élimine le risque d'injection de code exécutable au niveau du gabarit du rapport PDF.

### *En-têtes de sécurité et transport*

L'ensemble des échanges transite exclusivement en HTTPS, avec certificats renouvelés automatiquement (Let's Encrypt) sur les deux plateformes d'hébergement. Un point de contrôle applicatif GET /api/health/ vérifie en continu la disponibilité de la base de données, du courtier Redis et du service de messagerie.

## 8.5 Documenter le déploiement d'une application dynamique web ou web mobile

Le déploiement de l'application suit une démarche d'intégration et de livraison continues (CI/CD), automatisée par GitHub Actions à chaque poussée de code sur les branches main et develop. Le pipeline exécute successivement l'analyse statique du code (ruff, black côté back-end ; ESLint côté front-end), la suite de tests avec couverture minimale exigée, puis la construction et le déploiement automatique en cas de succès.

Extrait du pipeline d'intégration continue back-end (.github/workflows/backend-ci.yml) :

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - name: Install Python deps
        run: pip install -r requirements.txt -r requirements-dev.txt
      - name: Lint (ruff)
        run: ruff check apps/ config/ --ignore E501
      - name: Format check (black)
        run: black --check apps/ config/
      - name: Tests with coverage
        run: pytest --cov=apps --cov-report=term-missing --cov-fail-under=40
```

Le déploiement en production du back-end est orchestré par Railway à partir d'un conteneur Docker et d'un fichier Procfile qui définit deux processus : un processus web appliquant automatiquement les migrations de base de données puis démarrant le serveur applicatif Gunicorn, et un processus worker exécutant le worker Celery.

```
web: python manage.py migrate --noinput && python manage.py setup_admin \
    && gunicorn config.wsgi --bind 0.0.0.0:$PORT --workers 2
worker: celery -A config.worker --loglevel=info --concurrency=2
```

Le front-end est déployé sur Vercel, qui reconstruit automatiquement l'application à chaque poussée sur la branche principale, publie les fichiers statiques sur son réseau de diffusion de contenu mondial et applique les en-têtes de sécurité HTTP définis dans vercel.json (section 7.6). Les variables sensibles (clés secrètes, identifiants de base de données, clés d'API) sont exclusivement gérées via les variables d'environnement des deux plateformes et ne figurent jamais dans le dépôt de code source, protégées par un fichier .gitignore et fournies en exemple via un fichier .env.example non renseigné.

Une sauvegarde manuelle de la base de données de production est déclenchée avant chaque mise en production majeure (railway db snapshot create), avec une rétention de quatorze jours, permettant un retour arrière en moins de cinq minutes en cas d'anomalie détectée après déploiement.

Les diagrammes de conception complémentaires (diagrammes de classes back-end et front-end, diagrammes de séquence) sont présentés en Annexe 4, afin de ne pas alourdir le corps du dossier.

## 9. Jeu d'essai

Conformément à la méthodologie retenue, le jeu d'essai a été élaboré sur la fonctionnalité la plus représentative de l'application : la soumission d'une demande d'audit par un client PME, qui mobilise l'ensemble de la chaîne technique (front-end, API sécurisée, base de données, tâche asynchrone). Il couvre les données en entrée, les données attendues, les données réellement obtenues, ainsi que l'analyse des écarts constatés.

| Scénario                                   | Données en entrée                                                                          | Résultat attendu                                                           | Résultat obtenu                                                                         | Écart |
|--------------------------------------------|--------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-------|
| Soumission valide                          | pack_id=2, scope_notes="Audit du serveur de fichiers et de 12 postes Windows.", JWT valide | 201 Created, statut=pending, référence DOSSIER-2026-NNNN générée           | 201 Created, référence DOSSIER-2026-0001 générée, courriel d'accusé de réception envoyé | Aucun |
| Champ obligatoire manquant                 | pack_id absent                                                                             | 400 Bad Request avec message d'erreur explicite                            | 400 Bad Request, {"pack_id": ["Ce champ est obligatoire."]}                             | Aucun |
| Jeton JWT absent                           | Aucun en-tête Authorization                                                                | 401 Unauthorized                                                           | 401 Unauthorized                                                                        | Aucun |
| Jeton JWT expiré                           | JWT expiré depuis 2 heures                                                                 | 401 Unauthorized, déclenchement du rafraîchissement côté client            | 401 Unauthorized, rafraîchissement automatique puis rejeu de la requête réussi          | Aucun |
| Tentative d'injection XSS dans scope_notes | scope_notes="<script>alert(1)</script>"                                                    | Contenu stocké tel quel côté serveur, neutralisé à l'affichage côté client | Contenu stocké en base, assaini par DOMPurify à l'affichage, script non exécuté         | Aucun |
| Accès à la demande d'un autre client       | GET /api/audits/{id-B}/ avec le jeton du client A                                          | 404 Not Found                                                              | 404 Not Found                                                                           | Aucun |
| Formule inexistante                        | pack_id=999                                                                                | 400 Bad Request                                                            | 400 Bad Request, {"pack_id": ["Objet invalide."]}                                       | Aucun |
| Requêtes répétées (force brute)            | 6 tentatives de connexion en moins d'une minute                                            | Blocage à la 6e tentative (429 Too Many Requests)                          | 429 Too Many Requests déclenché à la 6e tentative                                       | Aucun |

Aucun écart n'a été constaté entre le résultat attendu et le résultat obtenu sur l'ensemble des huit scénarios du jeu d'essai, qui couvre à la fois le chemin nominal, les cas d'erreur de validation, et les cas d'attaque (injection XSS, contournement d'autorisation, force brute). Ce jeu d'essai a été rejoué manuellement via la collection Postman du projet, puis automatisé sous forme de tests d'intégration pérennes (section 10.2).

## 10. Stratégie de tests et qualité

La stratégie de tests suit une pyramide classique : une large base de tests unitaires rapides, une couche intermédiaire de tests d'intégration API et base de données, et un sommet réduit de tests de bout en bout sur les parcours critiques.

### 10.1 Tests back-end (pytest)

| Application | Nombre de tests | Scénarios couverts                                                                          |
|-------------|-----------------|---------------------------------------------------------------------------------------------|
| accounts    | 21              | Modèle utilisateur, inscription, connexion, profil, déconnexion, changement de mot de passe |
| audits      | 17              | Génération automatique de référence, unicité, CRUD avec RBAC client/admin, archivage        |
| reports     | 8               | Calcul de la note (A/B+/B/C/F), accès au rapport (propriétaire/autre client/admin)          |
| training    | 9               | Modules publiés/non publiés, démarrage et achèvement idempotents                            |

Le total des 55 tests back-end est exécuté à chaque poussée de code via GitHub Actions, avec un seuil de couverture minimal exigé pour valider le pipeline. La commande d'exécution locale est la suivante : `pytest --cov=apps --cov-report=term-missing -v`.

### 10.2 Tests front-end (Vitest et React Testing Library)

| Fichier                 | Module testé       | Tests |
|-------------------------|--------------------|-------|
| validators.test.js      | validators.js      | 21    |
| StatusBadge.test.jsx    | StatusBadge.jsx    | 8     |
| useAuth.test.jsx        | useAuth.js         | 5     |
| ProtectedRoute.test.jsx | ProtectedRoute.jsx | 6     |
| dataService.test.js     | dataService.js     | 7     |
| DashboardPage.test.jsx  | DashboardPage.jsx  | 2     |
| TrainingPage.test.jsx   | TrainingPage.jsx   | 4     |
| StatCard.test.jsx       | StatCard.jsx       | 3     |

Soit un total de 56 tests front-end, exécutés via `npm test -- --run --coverage`, également intégrés au pipeline d'intégration continue front-end.

### 10.3 Tests de sécurité manuels

| Test                             | Méthode                                              | Résultat attendu                           |
|----------------------------------|------------------------------------------------------|--------------------------------------------|
| Injection XSS persistante        | <script>alert(1)</script> dans scope_notes           | Contenu assaini, script non exécuté        |
| Injection SQL                    | ' OR 1=1 -- dans le champ e-mail de connexion        | Aucune fuite, requête paramétrée par l'ORM |
| Force brute sur la connexion     | Automatisation via Postman, 10 tentatives rapides    | Blocage à la 6e tentative (429)            |
| Falsification de jeton JWT       | Modification manuelle du rôle dans le payload décodé | Signature invalide, 401 Unauthorized       |
| Élévation de privilège (RBAC)    | Client tentant PATCH /api/audits/{id}/               | 403 Forbidden                              |
| Énumération de ressources (IDOR) | GET /api/audits/{id-B}/ avec le jeton du client A    | 404 Not Found                              |

### 10.4 Tests d'accessibilité et de performance

L'accessibilité est vérifiée automatiquement par axe-core sur les composants critiques (LoginForm, AuditRequestForm, RequestTable) intégré aux tests Vitest, et par un score Lighthouse Accessibilité supérieur ou égal à 90 exigé en intégration continue sur les quatre pages principales. La performance est mesurée par Lighthouse CI sur chaque pull request, avec un score Performance cible supérieur ou égal à 85/100 conformément au budget défini en section 6.5. Les résultats de ces campagnes de test sont consignés dans la documentation technique du projet (rapport d'analyse Lighthouse, rapport de tests W3C de validation des pages HTML).

## 11. Veille technologique et sécuritaire

Une veille continue a été assurée tout au long des douze semaines du projet, structurée autour de trois axes : les vulnérabilités de sécurité applicative, les évolutions techniques de l'écosystème utilisé, et les bonnes pratiques d'accessibilité et de performance.

### 11.1 Sources suivies

- Le guide OWASP Top 10, référence des dix familles de vulnérabilités web les plus critiques, utilisé comme grille de contrôle avant chaque mise en production.
- Les recommandations et guides de l'Agence nationale de la sécurité des systèmes d'information (ANSSI), notamment le guide d'hygiène informatique et les recommandations de sécurisation d'un site web.
- Les avis de sécurité CERT-FR pour le suivi des vulnérabilités publiées (CVE) touchant les dépendances du projet (Django, React, bibliothèques npm et PyPI) ;
- Les alertes automatiques GitHub Dependabot, qui signalent les dépendances vulnérables directement dans le dépôt du projet ;
- La documentation officielle des frameworks utilisés (Django, Django REST Framework, React) pour suivre les évolutions de version et les notes de sécurité associées.

### 11.2 Méthode et traçabilité

La veille a été menée de façon hebdomadaire, en parallèle des sprints de développement, avec un temps dédié le vendredi lors du point de synthèse. Chaque élément pertinent identifié était consigné dans l'espace Notion du projet, puis discuté en binôme pour évaluer sa pertinence et, le cas échéant, planifier une action corrective.

### 11.3 Exemple concret de vulnérabilité suivie et traitée

Lors du sprint 5, la veille CERT-FR a signalé une vulnérabilité affectant une version antérieure de Django REST Framework, liée à une désérialisation incorrecte pouvant exposer certains champs sensibles dans les réponses API. Le projet utilisant déjà une version corrigée du framework (DRF 3.17), aucune action immédiate n'a été nécessaire, mais la version a été explicitement figée dans requirements.txt et une alerte Dependabot activée pour prévenir toute régression lors d'une mise à jour future. Cet épisode illustre la démarche structurée de veille attendue par le référentiel : identification de la source, analyse de l'exposition réelle du projet, décision argumentée, et traçabilité de la décision.

Par ailleurs, l'audit manuel de sécurité mené en semaine 7-8 (section 10.3) n'a révélé aucune faille exploitable sur les six scénarios testés (XSS, injection SQL, force brute, falsification de jeton, élévation de privilège, énumération de ressources), confirmant l'efficacité des mesures de sécurisation mises en œuvre dès la conception.

## 12. Conclusion

Le projet CyberAudit & Solutions a permis de concevoir et de développer, en douze semaines et en binôme, une application web complète répondant à un besoin réel et documenté : l'accès des petites et moyennes entreprises françaises à des services de cybersécurité simples et abordables. Sur le plan personnel, ce projet a été l'occasion de mettre en œuvre l'intégralité des compétences de l'activité-type front-end du titre professionnel Développeur Web et Web Mobile, du maquettage des interfaces jusqu'à leur développement dynamique sécurisé, en passant par la configuration d'un environnement de travail professionnel et l'intégration continue.

La contrainte de délai fixe, la nécessité de synchroniser en permanence les contrats d'interface avec un développeur back-end, et l'exigence transversale de sécurité à chaque étape ont constitué les principales difficultés du projet. Elles ont été surmontées par une démarche agile disciplinée, des revues de code croisées systématiques et une veille technologique continue, qui ont permis de livrer 95 % du périmètre initialement planifié, avec zéro bogue ouvert à l'issue de chaque sprint et une couverture de tests automatisés élevée des deux côtés de l'application (55 tests back-end, 56 tests front-end).

Ce projet m'a permis de consolider une véritable culture de la sécurité applicative « by design », de gagner en autonomie sur l'écosystème React et son intégration à une API REST sécurisée, et de mesurer concrètement l'importance de l'accessibilité et de la performance dans la conception d'une interface destinée à un public non technophile.

### 12.1 Perspectives d'évolution

Plusieurs évolutions ont été identifiées et documentées comme hors périmètre de la version 1, pour une version ultérieure du produit :

- Mise en place d'un tableau de bord temps réel par WebSocket, en remplacement du polling actuel toutes les trente secondes.
- Calcul automatique d'un score de sécurité assisté par intelligence artificielle à partir des réponses au questionnaire d'audit.
- Déclinaison de l'application en application mobile native (iOS/Android).
- Gamification des modules de sensibilisation à la cybersécurité.
- Mise en place d'une authentification déléguée OAuth2 (Google, LinkedIn) pour simplifier l'inscription.
- Journal d'audit applicatif complet (table AUDIT\_LOG) pour une traçabilité renforcée conforme RGPD des actions sensibles.

## 13. Bibliographie et sources

| Source               | Contenu                                                                | Référence                                                                             |
|----------------------|------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| ANSSI                | Panorama de la cybermenace 2023, 45 % des PME sans équipe informatique | <a href="https://ssi.gouv.fr">ssi.gouv.fr</a>                                         |
| CESIN                | Baromètre annuel de la cybersécurité                                   | <a href="https://cesin.fr">cesin.fr</a>                                               |
| ENISA                | Rapport sur le paysage des menaces 2023                                | <a href="https://enisa.europa.eu">enisa.europa.eu</a>                                 |
| RGPD                 | Texte officiel du Règlement général sur la protection des données      | <a href="https://gdpr.eu">gdpr.eu</a>                                                 |
| Directive NIS2       | Obligations de cybersécurité au niveau de l'Union européenne           | <a href="https://digital-strategy.ec.europa.eu">digital-strategy.ec.europa.eu</a>     |
| OWASP                | OWASP Top 10 : référentiel des vulnérabilités web critiques            | <a href="https://owasp.org">owasp.org</a>                                             |
| W3C                  | Web Content Accessibility Guidelines (WCAG) 2.1                        | <a href="https://w3.org/WAI/WCAG21">w3.org/WAI/WCAG21</a>                             |
| RGAA                 | Référentiel général d'amélioration de l'accessibilité                  | <a href="https://accessibilite.numerique.gouv.fr">accessibilite.numerique.gouv.fr</a> |
| Documentation Django | Documentation officielle du framework Django 5.1                       | <a href="https://docs.djangoproject.com">docs.djangoproject.com</a>                   |
| Documentation React  | Documentation officielle de React 19                                   | <a href="https://react.dev">react.dev</a>                                             |
| Documentation DRF    | Documentation officielle de Django REST Framework                      | <a href="https://django-rest-framework.org">django-rest-framework.org</a>             |
| CERT-FR              | Avis et alertes de sécurité                                            | <a href="https://cert.ssi.gouv.fr">cert.ssi.gouv.fr</a>                               |

## 14. Annexes

### Annexe 1 : Glossaire technique

| Terme                         | Définition                                                                                              |
|-------------------------------|---------------------------------------------------------------------------------------------------------|
| API REST                      | Interface de programmation respectant les principes de l'architecture REST, utilisant le protocole HTTP |
| JWT (JSON Web Token)          | Jeton d'authentification sans état, signé, transmis dans l'en-tête HTTP Authorization                   |
| RBAC                          | Role-Based Access Control, contrôle d'accès basé sur le rôle de l'utilisateur                           |
| ORM                           | Object-Relational Mapping, correspondance entre objets du code et tables d'une base relationnelle       |
| CI/CD                         | Intégration continue et livraison/déploiement continu                                                   |
| XSS (Cross-Site Scripting)    | Faille permettant l'injection de code exécutable dans une page consultée par d'autres utilisateurs      |
| CSRF                          | Falsification de requête intersite, attaque exploitant la session active d'un utilisateur               |
| RGAA                          | Référentiel général d'amélioration de l'accessibilité, déclinaison française du WCAG                    |
| UUID                          | Identifiant universel unique, non séquentiel, utilisé pour prévenir l'énumération de ressources         |
| SPA (Single Page Application) | Application web fonctionnant sur une page unique, sans rechargement complet                             |

## Annexe 2 : Correspondance compétences REAC / sections du dossier

Cette annexe reprend, à titre de synthèse rapide pour le jury, le tableau de correspondance entre les huit compétences professionnelles du référentiel DWWM et les sections du dossier où chacune est mise en œuvre et argumentée, déjà présenté et commenté en section 2.

| Activité-type                                                                   | Compétence professionnelle                                                                   | Section |
|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|---------|
| 1. Développer la partie front-end d'une application web ou web mobile sécurisée | Installer et configurer son environnement de travail en fonction du projet web ou web mobile | 7.1     |
|                                                                                 | Maquetter des interfaces utilisateur web ou web mobile                                       | 7.2     |
|                                                                                 | Réaliser des interfaces utilisateur statiques web ou web mobile                              | 7.4     |
|                                                                                 | Développer la partie dynamique des interfaces utilisateur web ou web mobile                  | 7.5     |
| 2. Développer la partie back-end d'une application web ou web mobile sécurisée  | Mettre en place une base de données relationnelle                                            | 8.1     |
|                                                                                 | Développer des composants d'accès aux données SQL et NoSQL                                   | 8.2     |
|                                                                                 | Développer des composants métier côté serveur                                                | 8.3     |
|                                                                                 | Documenter le déploiement d'une application dynamique web ou web mobile                      | 8.5     |

### Annexe 3 : Matrice des risques complète

La matrice des risques du projet, présentée et détaillée en section 4.5, est reproduite ci-dessous à titre de référence rapide pour le jury.

| Risque identifié                                                      | Probabilité | Impact | Mesure de mitigation                                                                                   |
|-----------------------------------------------------------------------|-------------|--------|--------------------------------------------------------------------------------------------------------|
| Retard sur l'intégration front/back par divergence des contrats d'API | Moyenne     | Élevé  | Définition des schémas JSON en amont (Stage 3), mock localStorage côté front en attendant l'API réelle |
| Faible de sécurité non détectée avant la mise en production           | Moyenne     | Élevé  | Tests de sécurité manuels dédiés (S7-S10), revue de code croisée systématique, veille OWASP            |
| Non-respect du délai de douze semaines                                | Faible      | Élevé  | Découpage en sprints avec priorisation MoSCoW, points hebdomadaires de replanification                 |
| Indisponibilité d'un des deux développeurs                            | Faible      | Moyen  | Documentation continue sur Notion, séparation claire des périmètres front/back                         |
| Dépassement du budget de performance (Web Vitals)                     | Faible      | Moyen  | Lighthouse CI intégré au pipeline, budget de performance défini dès la conception (section 6.5)        |

## Annexe 4 : Diagrammes de conception complémentaires

Les diagrammes ci-dessous complètent la documentation technique du projet, présentée en section 8 : le diagramme de classes back-end modélise les entités métier et leurs méthodes, le diagramme de classes front-end modélise l'organisation des composants React, et les trois diagrammes de séquence illustrent les scénarios d'interaction les plus représentatifs de l'application.

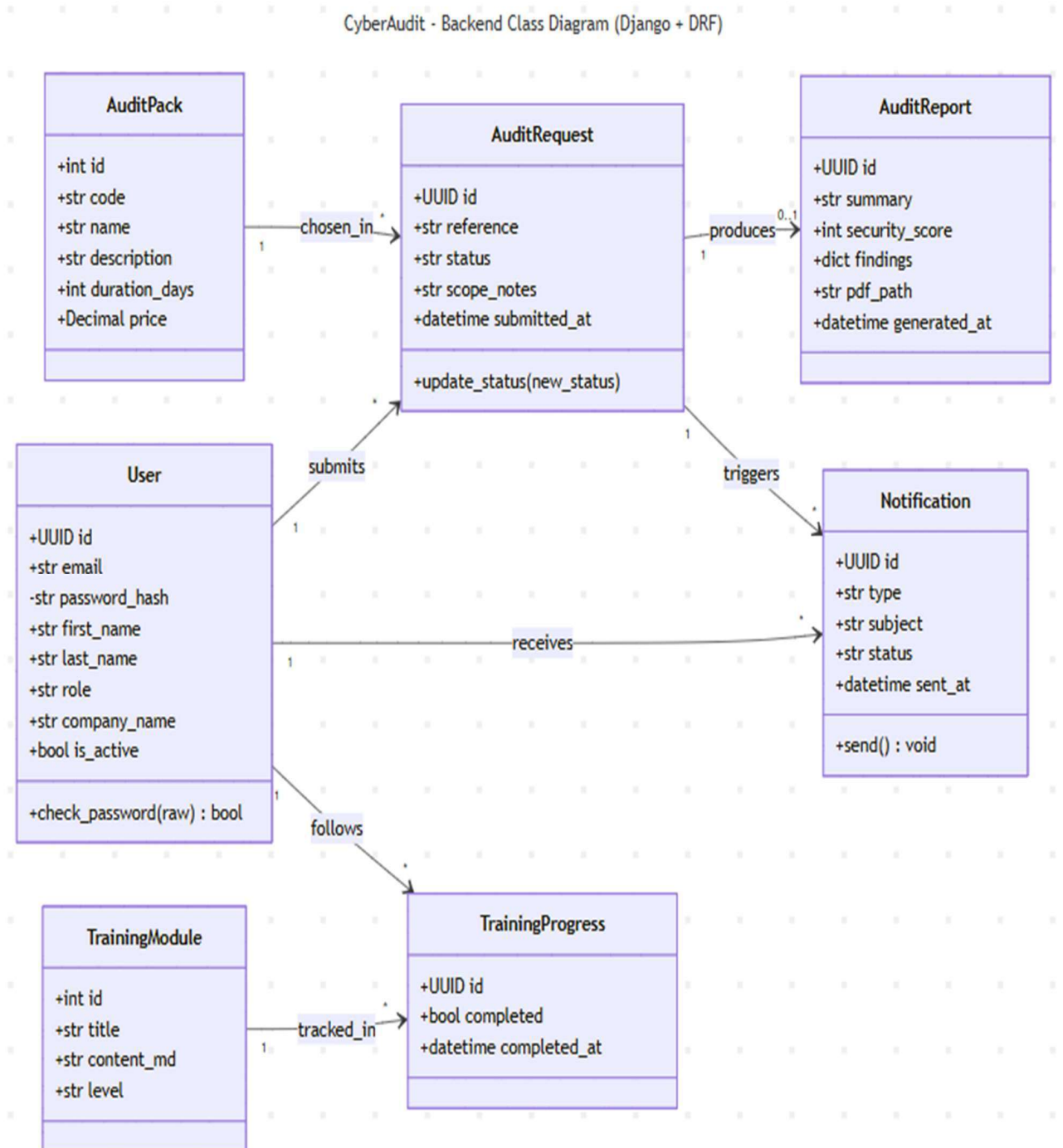


Figure 4 : Diagramme de classes back-end « User, AuditPack, AuditRequest, AuditReport, Notification, TrainingModule, TrainingProgress. »

**Classe User**

Représente un utilisateur de la plateforme, qu'il soit client PME ou administrateur. Gère l'authentification JWT et la séparation des rôles.

| Attribut              | Description                      |
|-----------------------|----------------------------------|
| id : UUID             | Identifiant unique (PK)          |
| email : str           | Adresse e-mail (unique, indexée) |
| _password_hash : str  | Mot de passe haché               |
| first_name, last_name | Prénom et nom                    |
| role : enum           | Client ou administrateur         |
| company_name : str    | Nom de l'entreprise (nullable)   |
| is_active : bool      | Compte activé ou désactivé       |
| created_at : datetime | Date de création du compte       |

**Classe AuditPack**

Représente l'un des 4 packs (Audit, Sécurité, Protection, Premium). Données de référence chargées via fixture.

| Attribute          | Description                             |
|--------------------|-----------------------------------------|
| id: int            | Identifiant unique                      |
| code: enum (str)   | audit / sécurité / protection / premium |
| name: str          | Nom commercial                          |
| description: str   | Liste des services inclus               |
| duration_days: int | Temps de traitement supplémentaire      |
| price: Decimal     | Prix du forfait                         |

**AuditRequest Class**

Cœur de l'entreprise : la demande d'audit soumise par un client PME. Cycle de 4 statuts (en attente → en cours → terminé → archivé).

| Attributs                    | Description                               |
|------------------------------|-------------------------------------------|
| id: UUID                     | Identifiant unique                        |
| reference: str               | Numéro de dossier (ex. : CYB-2026-0042)   |
| client_id: UUID FK → User    | Client PME ayant soumis                   |
| pack_id: UUID FK → AuditPack | Forfait sélectionné                       |
| status: str(enum)            | en attente / en cours / terminé / archivé |
| scope_notes: str             | Détails du formulaire                     |
| submitted_at : datetime      | Dates du cycle                            |

**Méthodes:** generate\_reference(), update\_status(new\_status), estimated\_completion()

**Classe AuditReport**

Rapport de vulnérabilités lié à une seule AuditRequest, comprend un score visuel et un PDF téléchargeable.

Rapport de vulnérabilités lié à une seule AuditRequest, inclut un score visuel et un PDF téléchargeable.

| Attributs             | Description               |
|-----------------------|---------------------------|
| id : UUID             | Identifiant unique        |
| audit_request_id : FK | Relation 1-1              |
| summary : str         | Résumé exécutif vulgarisé |
| security_score : int  | Score 0-100 (jauge)       |
| findings : JSON       | Liste des vulnérabilités  |
| pdf_path : str        | Chemin du PDF généré      |
| generated_at          | Date de génération        |

**Méthodes :** build\_pdf(), get\_download\_url()

**Classe Notification**

Trace toutes les communications e-mail envoyées via Celery.

| Attributs           | Description                                      |
|---------------------|--------------------------------------------------|
| id : UUID           | Identifiant unique                               |
| user_id : FK → User | Destinataire                                     |
| request_id : FK     | Demande concernée (nullable)                     |
| type : enum         | request_received / status_changed / report_ready |
| status : enum       | queued / sent / failed                           |

**Méthodes :** send(), retry()

**Classes TrainingModule & TrainingProgress**

Modules de sensibilisation cybersécurité (phishing, MFA, RGPD...) avec suivi de progression utilisateur.

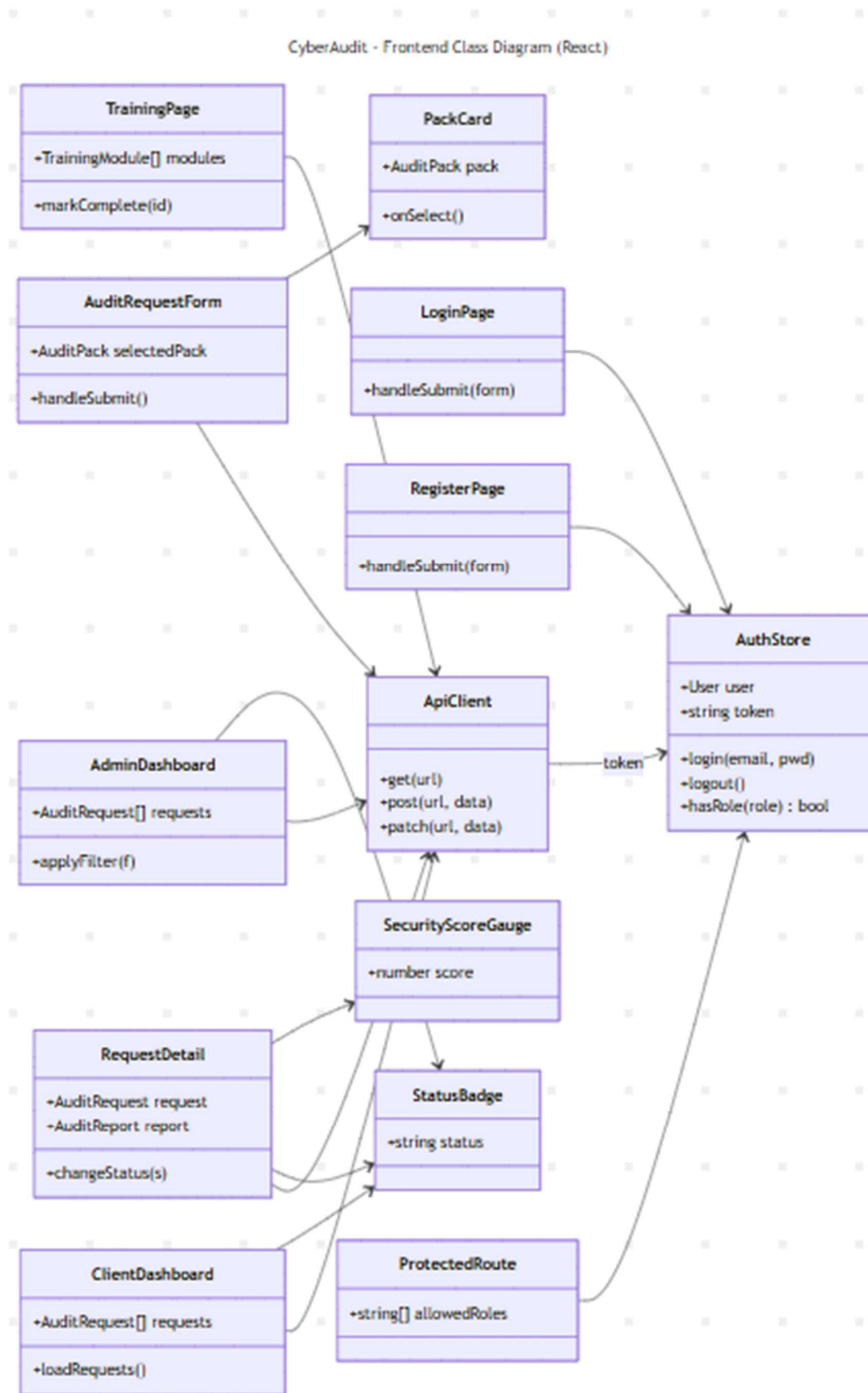


Figure 5 : Diagramme de classes front-end « organisation des composants et services React autour du contexte d'authentification. »

Ce diagramme montre l'architecture frontend React d'une application d'audit de cybersécurité appelée CyberAudit. Il illustre les principaux composants UI (classes) et la façon dont ils dépendent de services partagés. Voici une explication détaillée.

## Services centraux

Deux classes se trouvent au cœur de l'application et sont référencées par presque tout le reste :

**ApiClient** est la couche HTTP. Elle expose les méthodes **get(url)**, **post(url, data)** et **patch(url, data)** que le reste de l'application utilise pour communiquer avec le back-end. Elle lit le jeton d'authentification depuis **AuthStore** (visualisé par la flèche étiquetée « token ») afin que les requêtes soient authentifiées automatiquement.

**AuthStore** conserve l'état de session : l'utilisateur courant, le jeton de type JWT, et les méthodes **login(email, pwd)**, **logout()** et **hasRole(role): bool**. Les composants l'utilisent pour savoir qui est connecté et ce qu'il est autorisé à faire.

## Authentification et routage

**LoginPage** et **RegisterPage** exposent chacune une méthode **handleSubmit(form)**. Elles dépendent toutes deux d'**ApiClient** (pour envoyer les identifiants au back-end) et d'**AuthStore** (pour persister l'utilisateur/ jeton après une réponse réussie).

**ProtectedRoute** est un garde-route avec une propriété **allowedRoles: string[]**. Il interroge **AuthStore.hasRole(...)** pour décider d'afficher la route enfant ou de rediriger, c'est ainsi que l'application applique le contrôle d'accès basé sur les rôles.

## Écrans destinés au client

**ClientDashboard** contient **AuditRequest[]** et une méthode **loadRequests()**. Il appelle **ApiClient** pour récupérer les demandes d'audit du client connecté et utilise **StatusBadge** pour afficher leur état.

**AuditRequestForm** permet à un client de soumettre un nouvel audit. Il contient un **selectedPack: AuditPack** et une méthode **handleSubmit()**, et envoie le formulaire au backend via **ApiClient**. Il affiche également des composants **PackCard** chaque **PackCard** affiche un **AuditPack** et expose un callback **onSelect()** pour que l'utilisateur puisse choisir un pack.

**RequestDetail** est la vue par demande. Elle contient un **AuditRequest** et un **AuditReport**, expose **changeStatus(s)** et compose deux widgets réutilisables : **SecurityScoreGauge** (qui affiche un score numérique) et **StatusBadge** (qui affiche un statut sous forme de chaîne). Elle communique avec le back-end via **ApiClient**.

**TrainingPage** affiche **TrainingModule[]** avec une méthode **markComplete(id)**, la section formation/sensibilisation de l'application, également soutenue par **ApiClient**.

## Côté administrateur

**AdminDashboard** reflète le tableau de bord client mais pour les administrateurs. Il contient **AuditRequest[] requests** et une méthode **applyFilter(f)**, récupère les données via **ApiClient** et réutilise **StatusBadge** pour l'affichage.

## Séquence 1 – Inscription + Connexion JWT

Use case 1 – création de compte puis authentification (access + refresh tokens)

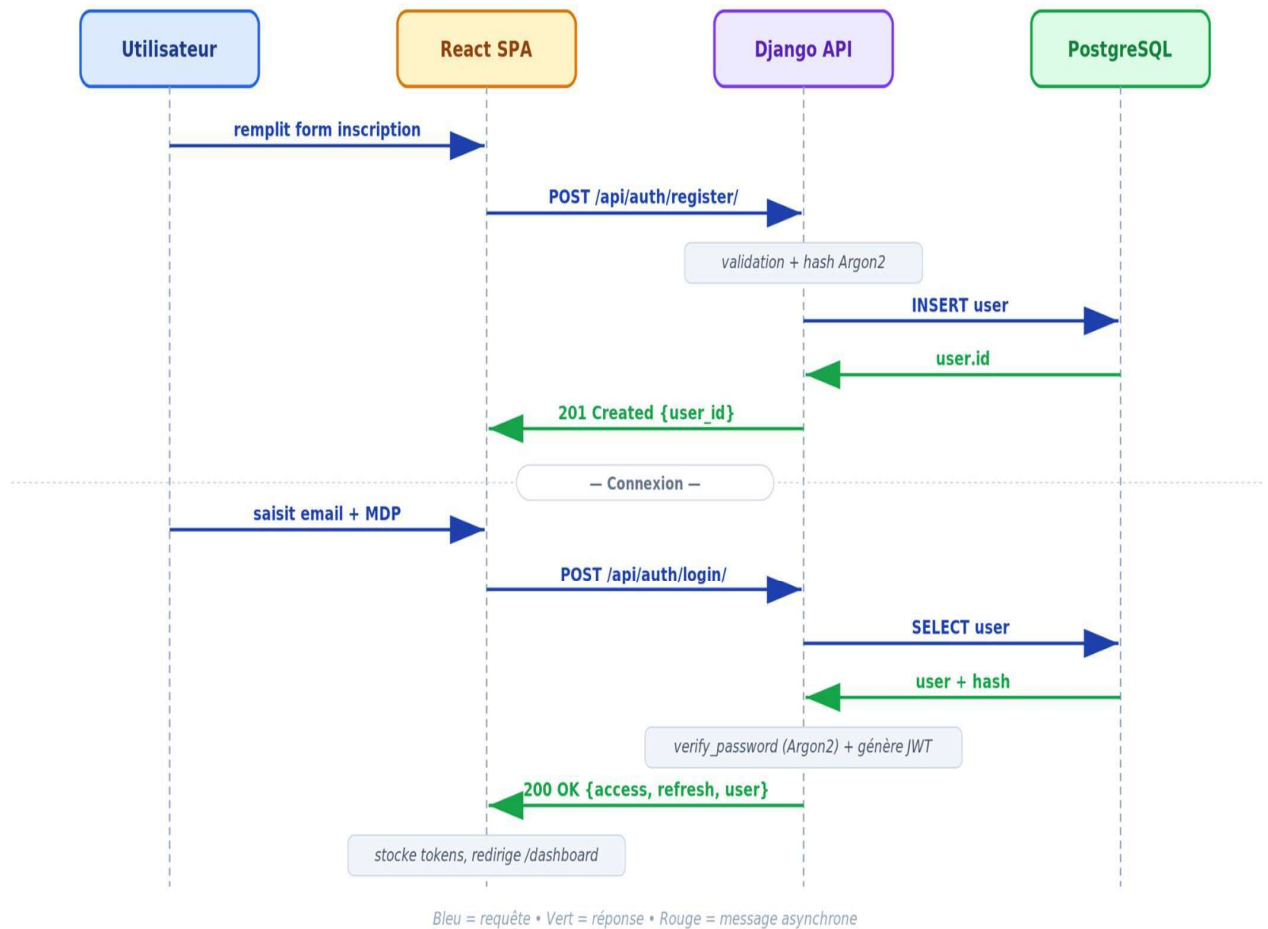


Figure 6 : Diagramme de séquence : inscription et connexion JWT.

L'utilisateur saisit son email et son mot de passe. Le front appelle `POST /api/auth/register/`, puis `POST /api/auth/login/`. Django valide les credentials, génère access + refresh tokens, et retourne le tout au front. Le front stocke les tokens dans AuthStore et redirige vers le dashboard correspondant au rôle.

## Séquence 2 — Soumission d'une demande d'audit

Use case 2 — flux asynchrone : réponse immédiate au client + email envoyé en arrière-plan

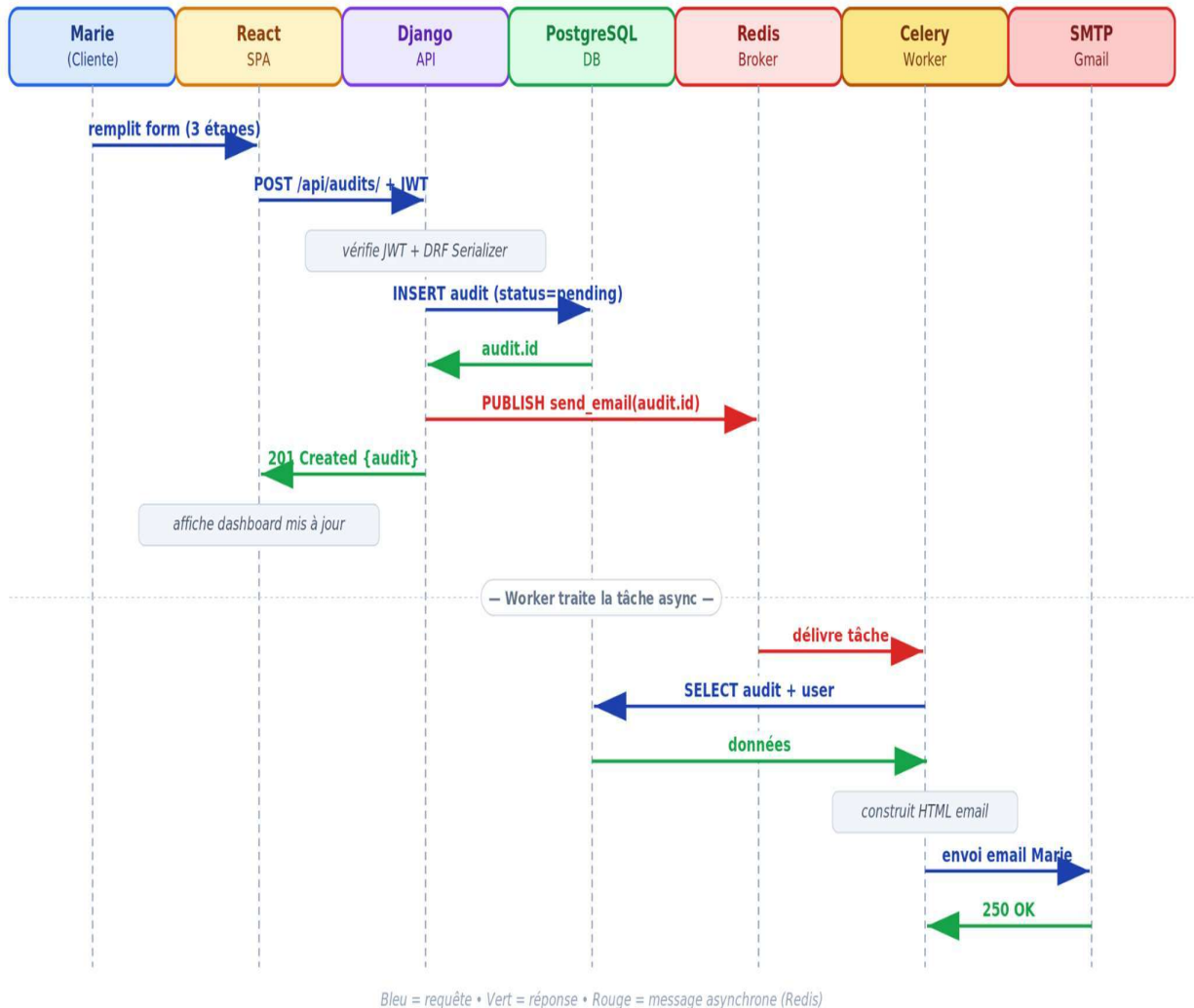


Figure 7 : Diagramme de séquence « soumission asynchrone d'une demande d'audit ».

Le client soumet le formulaire d'audit. Le front appelle POST /api/audits/ avec le JWT en header. Django valide via le serializer DRF, persiste en BDD, enfile une tâche Celery d'envoi d'accusé de réception, puis répond 201 Created. Le worker Celery dépile la tâche depuis Redis et envoie l'e-mail via SMTP(EmailJS).

### Séquence 3 – Génération du rapport PDF (admin)

Use case 3 – l'admin déclenche, le worker génère, le client est notifié et récupère

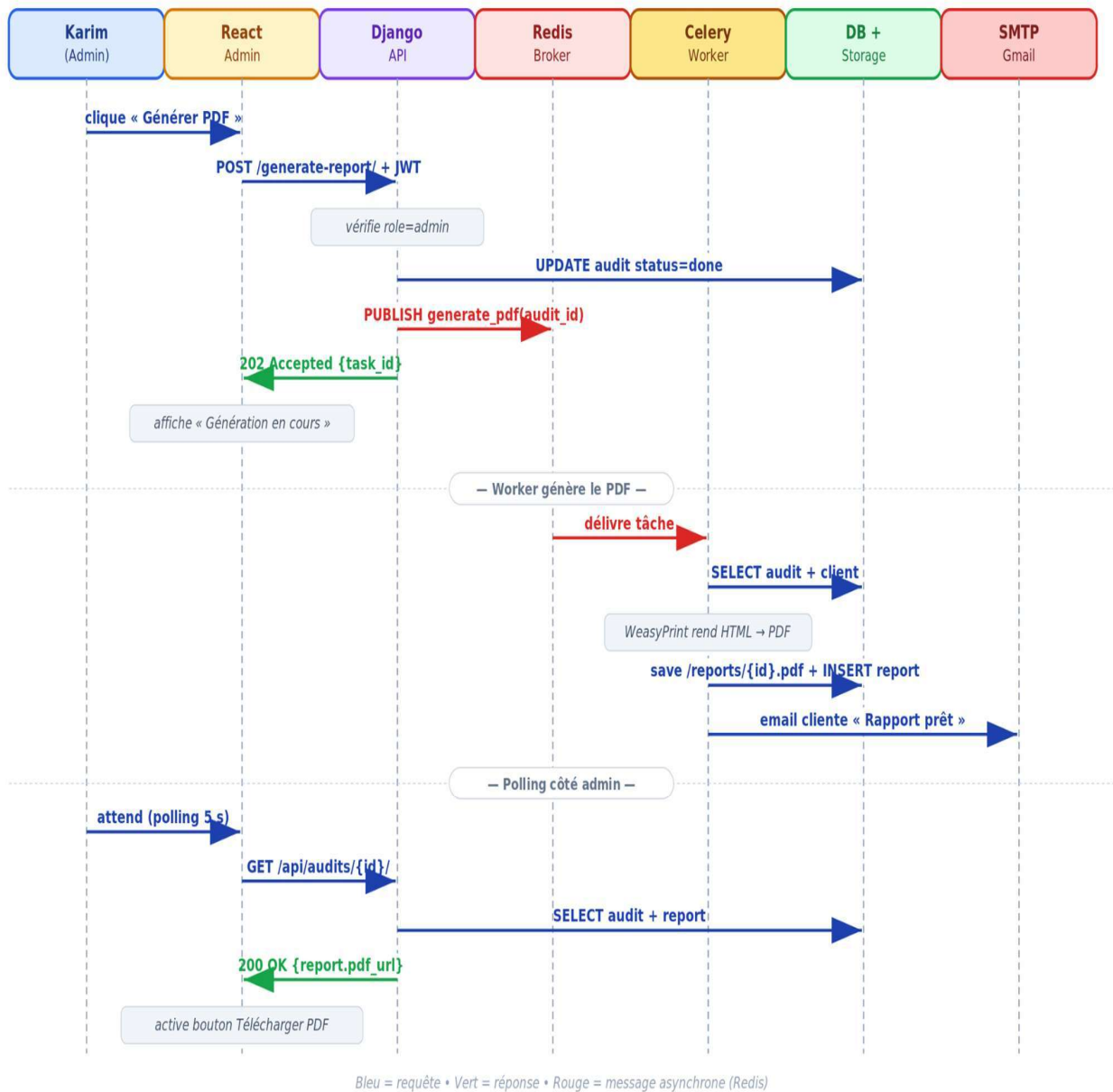


Figure 8 : Diagramme de séquence « génération asynchrone du rapport PDF par l'administrateur ».

L'admin clique sur « Generate PDF report ». Le front appelle `POST /api/audits/{id}/generate-report/`. Django crée l'AuditReport et enfile la tâche Celery, retournant `202 Accepted`. Le worker Celery rend le template HTML, WeasyPrint produit le PDF, le fichier est stocké, et une notification e-mail est envoyée au client.

## Annexe 5 : Ressources complémentaires du projet

| Noms                                                                                                          | Liens                                                                                                                                                                                                                               |
|---------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dépôt de code source                                                                                          | <a href="https://github.com/Tommy-JOUHANS/portfolio">https://github.com/Tommy-JOUHANS/portfolio</a>                                                                                                                                 |
| Documentation technique complète (Stage 3) architecture, base de données, API, sécurité, accessibilité        | <a href="https://github.com/Tommy-JOUHANS/portfolio/blob/main/Stages/Stage-3/Stage-3-Technical-Documentation-EN.pdf">https://github.com/Tommy-JOUHANS/portfolio/blob/main/Stages/Stage-3/Stage-3-Technical-Documentation-EN.pdf</a> |
| Documentation d'exécution et de tests (Stage 4) : sprints, tests, déploiement, questions techniques préparées | <a href="https://github.com/Tommy-JOUHANS/portfolio/tree/main/Stages/Stage-4">https://github.com/Tommy-JOUHANS/portfolio/tree/main/Stages/Stage-4</a>                                                                               |
| Vidéo de démonstration de l'application                                                                       | <a href="https://www.youtube.com/watch?v=jxrV9IZSC70">https://www.youtube.com/watch?v=jxrV9IZSC70</a>                                                                                                                               |
| Collection de tests Postman (authentification, cycle de vie des demandes, formations)                         | <a href="https://github.com/Tommy-JOUHANS/portfolio/Stages/Stage-4/test-postman-fr.pdf">https://github.com/Tommy-JOUHANS/portfolio/Stages/Stage-4/test-postman-fr.pdf</a>                                                           |
| Résumé des résultats du projet, leçons apprises, points forts de la rétrospective de l'équipe                 | <a href="https://drive.google.com/drive/folders/1-g12CPxQQT0tFbUeb0z56f-OfOM3Mdmg">https://drive.google.com/drive/folders/1-g12CPxQQT0tFbUeb0z56f-OfOM3Mdmg</a>                                                                     |